FIELD GUIDE · OCTOBER 2026

JUNIT 6 · MOCKITO 5 · ASSERTJ 3 · JAVA 21

```
$ mvn test
└─ JUnit Jupiter ✔
   ├─ OrderServiceTest ✔
   │  ├─ chargesDiscountedTotalSavesAndEmails() ✔
   │  └─ declinedPaymentSavesNothingAndSendsNoEmail() ✔
   └─ DiscountPolicyTest ✔
      └─ ₹1000 → 5% off ✔
[        44 tests successful      ]
[         0 tests failed          ]
```

The complete reference to

Testing *Java* applications

Unit, integration, API, end-to-end, performance and security testing — with JUnit 6, Mockito and AssertJ. Every code example in this guide was compiled and run, and every output shown is real.

`static` `unit` `integration` `contract` `end-to-end` `performance` `security`

`dummy` `stub` `spy` `mock` `fake`

6	44	5	1
chapters	tests, all run	test doubles	cheat sheet

INSIDE

Contents

01 Foundations

Why we test, how much of each kind, and how to choose test cases

- Why we test & the kinds of tests 03
- Pyramid, trophy & what makes a good test 04
- Designing test cases 05

02 Unit testing with JUnit 6

Annotations, lifecycle, assertions and parameterized tests

- Setup, architecture & JUnit 6 06
- Your first test & the lifecycle 07
- Assertions & AssertJ 08
- Parameterized, nested & more 09

03 Test doubles & Mockito

Replacing collaborators — and when not to

- The five test doubles 10
- Mockito in practice 11
- Failure paths, call order & the API 12
- Spies, statics, errors & pitfalls 13

04 Integration & API testing

Real HTTP, Spring Boot slices, containers and flaky tests

- Testing a real HTTP API 14
- Spring Boot, Testcontainers & contracts 15
- The @Transactional test trap 16
- Time, randomness, threads & flaky tests 17

05 Beyond correctness

End-to-end, performance and security testing

- End-to-end & UI testing 18
- Performance testing 19
- Security & resilience testing 20

06 Quality & process

TDD, measuring test quality, CI and a self-check

- Test-driven development 21
- Coverage & mutation testing 22
- Mutation testing with PIT — for real 23
- PIT in your build and CI 24
- Coverage for unit and integration tests 25
- Finding duplicate & redundant tests 26
- Build plugins for test quality 27
- Parallel tests: threads vs JVMs 28
- Singleton containers per JVM 29
- Build tools & CI pipelines 30
- Team testing conventions 31
- Check your understanding 32

Cheat sheet — one page to pin above your desk 33

Testing checklist, references & how this guide was verified 34

Index A–Z 35

Versions and facts as of October 2026. Every entry, page reference and footer link is clickable.

COLOUR LANGUAGE

Each kind of test keeps one colour on every page. Test doubles have their own: `stub` `mock`

`static`	checked without running
`unit`	one class, in memory
`integration`	real parts together
`contract`	agreed API shape
`end-to-end`	the whole system
`performance`	speed under load
`security`	attacks & weaknesses

CALLOUTS

TIP
Practical advice you can apply today.

WATCH OUT
A mistake that shows up in real code bases.

RULE OF THUMB
A default that is right most of the time.

> *"Program testing can be used to show the presence of bugs, but never to show their absence!"*
>
> — Edsger W. Dijkstra

CHAPTER ONE

01 Foundations

A test is code that runs your code and checks the answer. Good tests let you change software quickly without breaking it; bad ones slow you down and still let bugs through. This chapter covers what to test, how much of each kind, and how to pick the cases.

What tests buy you

Fast feedback

Find a mistake seconds after you make it, not weeks later in production.

Safe change

Refactor or upgrade a library and know at once whether behaviour changed.

Living docs

A test named `declinedPaymentSavesNothing` states a rule that can't go out of date.

Better design

Code that is hard to test is usually too coupled. Tests push you to inject dependencies.

The kinds of tests

KIND	WHAT IT CHECKS	SCOPE	SPEED	JAVA TOOLS
`static`	Code style, likely bugs, types — without running anything	Whole code base	Seconds	javac -Xlint, Error Prone, SpotBugs, Checkstyle
`unit`	One class or function; collaborators replaced by doubles	Single process, no I/O	Milliseconds	JUnit, AssertJ, Mockito
`integration`	Your code with real neighbours: DB, HTTP, queue, framework	Process + real dependencies	Sub-second to seconds	Spring Boot test slices, Testcontainers, WireMock
`contract`	Consumer and provider agree on the API shape	Two services, one at a time	Seconds	Pact, Spring Cloud Contract
`end-to-end`	A user journey through the deployed system	Browser / client → all services	Seconds to minutes	Playwright, Selenium, REST Assured
`performance`	Latency, throughput and limits under load	Production-like environment	Minutes to hours	Gatling, JMeter, k6, JMH
`security`	Known weaknesses, vulnerable libraries, attack paths	Code, dependencies, running app	Minutes	ZAP, OWASP Dependency-Check, Jazzer

GOOD TO KNOW

Google's *Software Engineering at Google* sorts tests by **size** instead of name: **small** tests run in one process, **medium** tests on one machine (they may call localhost), **large** tests anywhere. Size predicts speed and flakiness better than labels like "integration", which teams define differently.

Vocabulary

SUT System under test — the class or system being tested.

Test double A stand-in for a real collaborator.

Test oracle How you know the expected answer.

Coverage Which code the tests executed.

Fixture Everything set up so a test can run.

Regression Something that used to work and broke.

Smoke test A quick check that the build basically works.

Mutation score Share of injected bugs the tests caught.

Assertion A check that fails the test if it is false.

Flaky test Passes and fails without any code change.

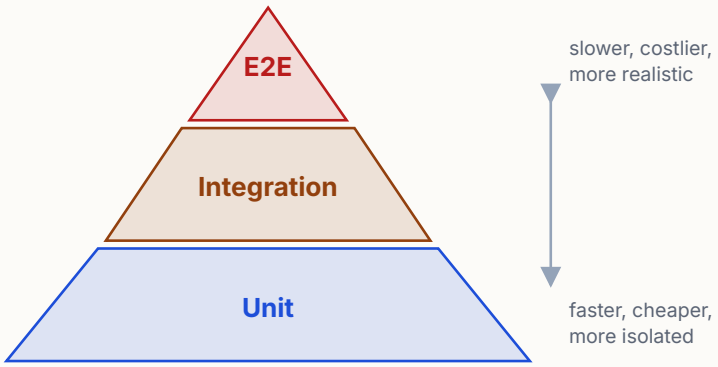
Acceptance test Checks a requirement from the user's view.

Test suite A group of tests run together.

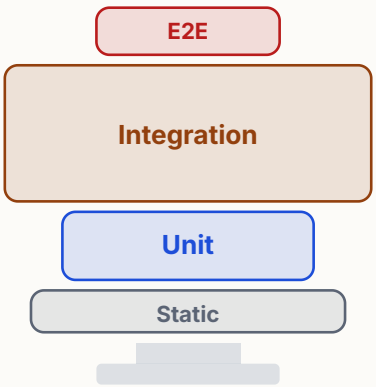
01 · FOUNDATIONS

Pyramid, trophy & what makes a good test

TEST PYRAMID · COHN, 2009



TESTING TROPHY · DODDS, 2018



The pyramid says: write many fast unit tests, fewer integration tests and only a handful of end-to-end tests, because cost and flakiness grow as you go up. It came from Mike Cohn's *Succeeding with Agile* (2009).

The trophy moves the weight to integration tests, arguing they give the most confidence per test, with static analysis as the base. Both agree on the real rule: **push each check down to the cheapest level that can catch the bug.**

FIRST: five properties of a good unit test

F	I	R	S	T
Fast	**Independent**	**Repeatable**	**Self-validating**	**Timely**
Milliseconds. Slow tests stop being run.	No shared state; any order, in parallel.	Same result anywhere: no real clock, network or randomness.	Pass or fail by itself; nobody reads logs.	Written with (or before) the code, not months later.

Acronym by Tim Ottinger and Brett Schuchert, popularised in Robert C. Martin's *Clean Code*.

Arrange · Act · Assert

MoneyTest.java ✔ passes

```
@Test
@DisplayName("10% off ₹999.99 rounds half-up to ₹899.99")
void percentOffRoundsHalfUp() {
  Money price = Money.of("999.99");  // Arrange

  Money discounted = price.percentOff(10);  // Act

  assertEquals(Money.of("899.99"), discounted); // Assert
}
```

Three blocks, separated by blank lines. **Given / When / Then** is the same idea in BDD words. If "Act" needs more than one line, the API under test is probably awkward.

Naming tests

STYLE	EXAMPLE
Behaviour sentence	`declinedPaymentSavesNothingAndSendsNoEmail`
method_state_result	`placeOrder_emptyItems_throws`
should…	`shouldRejectNegativeAmounts`
@DisplayName	`"10% off ₹999.99 rounds half-up"`

DO

- ✓ Test **behaviour** through the public API
- ✓ One reason to fail per test
- ✓ Make failures explain themselves

DON'T

- ✗ Assert on private fields or call order you don't care about
- ✗ Share mutable state between tests
- ✗ Put `if` or loops with logic in a test

01 · FOUNDATIONS

Designing test cases

You can't test every input, so pick the few that are most likely to find bugs. The rule: `DiscountPolicy` gives 0% below ₹1000, 5% from ₹1000 and 10% from ₹5000.

● = a value tested in DiscountPolicyTest — one inside each partition plus both sides of every boundary



DiscountPolicyTest.java

```
@ParameterizedTest(name = "₹{0} → {1}% off",
                   quoteTextArguments = false)
@CsvSource({
  "0,       0",
  "999.99,  0",   // just below the first boundary
  "1000,    5",   // on the boundary
  "1000.01, 5",
  "4999.99, 5",
  "5000,   10",   // on the second boundary
  "250000, 10"
})
```

test output · JUnit 6.1.3 — 7 / 7 passed

```
└ percentForSubtotal(String, int) ✔
   ├ ₹0 → 0% off ✔
   ├ ₹999.99 → 0% off ✔
   ├ ₹1000 → 5% off ✔
   ├ ₹1000.01 → 5% off ✔
   ├ ₹4999.99 → 5% off ✔
   ├ ₹5000 → 10% off ✔
   └ ₹250000 → 10% off ✔
```

Mistype `≥` as `>` and only the **₹1000** row fails — see the mutation demo on p.22. Middle-of-range values like ₹2000 would never notice.

Decision table for `placeOrder`

RULE	1	2	3	4
Items present?	no	yes	yes	yes
Payment approved?	—	no	yes	yes
Subtotal ≥ ₹1000?	—	—	no	yes
Outcome	reject, call nobody	throw, save nothing	save, email, 0%	save, email, 5–10%
Test	`emptyOrderIsRejected…`	`declinedPayment…`	`chargesBeforeSaving`	`chargesDiscounted…`

Test design techniques

TECHNIQUE	HOW	BEST FOR
Equivalence partitioning	Split inputs into groups the code treats the same; test one value from each.	Fewer tests, same coverage of behaviour
Boundary value analysis	Test on and either side of every edge: 999.99, 1000, 1000.01.	Catches off-by-one and `>` vs `≥`
Decision table	List every combination of conditions and the expected action.	Business rules with several inputs
State transition	Test each allowed move between states — and the forbidden ones.	Orders, payments, workflows
Pairwise	Cover every *pair* of option values instead of every combination.	Config and browser/OS matrices
Property-based	State a rule ("total ≥ 0") and let a tool generate hundreds of inputs.	Parsers, maths, serialisation (jqwik)
Error guessing	Use experience: null, empty, huge, Unicode, duplicates, time zones.	Finds what specs forget

Inputs worth trying on almost anything

Empty & missing
`null`, `""`, empty list, missing JSON field, zero items.

Extremes
`0`, `-1`, `Integer.MAX_VALUE`, 10 MB strings, 10,000 rows.

Text
Unicode (₹, emoji, Hindi), leading spaces, mixed case, SQL and HTML characters.

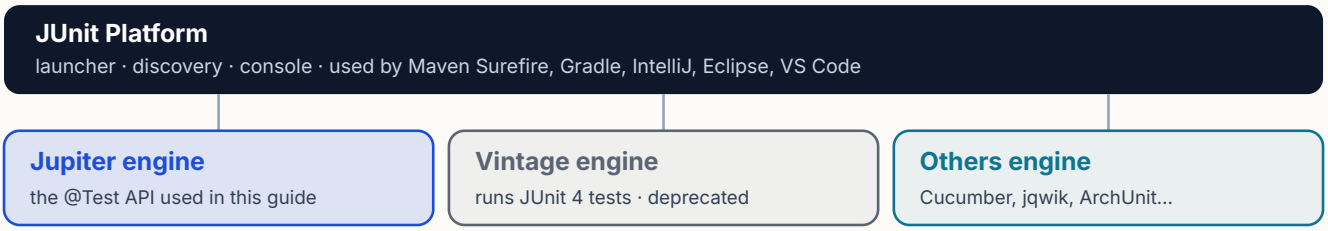
Time
Midnight, 29 Feb, month end, DST change, a different time zone.

CHAPTER TWO

02 Unit testing with JUnit 6

JUnit is the standard test framework for Java. Version 6 (30 September 2025) keeps the JUnit 5 programming model — the `org.junit.jupiter` API — and raises the floor to Java 17. Every test in this guide ran on JUnit 6.1.3, Mockito 5.24.0 and AssertJ 3.27.7 on Java 21.

How the pieces fit



Maven

pom.xml — test dependencies

```xml
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>6.1.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.27.7</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.24.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Gradle (Kotlin DSL)

build.gradle.kts

```kotlin
dependencies {
  testImplementation(platform("org.junit:junit-bom:6.1.3"))
  testImplementation("org.junit.jupiter:junit-jupiter")
  testRuntimeOnly("org.junit.platform:junit-platform-launcher")
  testImplementation("org.assertj:assertj-core:3.27.7")
  testImplementation("org.mockito:mockito-junit-jupiter:5.24.0")
}

tasks.test {
  useJUnitPlatform()
}
```

Coming from JUnit 4

JUNIT 4	JUNIT 5 / 6
`@Before / @After`	`@BeforeEach / @AfterEach`
`@BeforeClass / @AfterClass`	`@BeforeAll / @AfterAll`
`@Ignore`	`@Disabled`
`@Category(Slow.class)`	`@Tag("slow")`
`@RunWith(…)`	`@ExtendWith(…)`
`@Rule / @ClassRule`	`@ExtendWith or @RegisterExtension`
`@Test(expected = X.class)`	`assertThrows(X.class, …)`
`@Test(timeout = 500)`	`@Timeout(value = 500, unit = MILLISECONDS)`
`assertEquals(msg, exp, act)`	`assertEquals(exp, act, msg) — message last`
`public class / public void`	`package-private is enough`

`junit-jupiter` bundles the API, the parameterized-test module and the engine. `mockito-junit-jupiter` brings `mockito-core` with it. Run with `mvn test` or `./gradlew test`; one class with `mvn test -Dtest=MoneyTest`.

02 · UNIT TESTING WITH JUNIT 6

Your first test & the lifecycle

MoneyTest.java ✔ 3 / 3 passed

```java
class MoneyTest {

  @Test
  @DisplayName("10% off ₹999.99 rounds half-up to ₹899.99")
  void percentOffRoundsHalfUp() {
    Money price = Money.of("999.99");  // Arrange

    Money discounted = price.percentOff(10);  // Act

    assertEquals(Money.of("899.99"), discounted); // Assert
  }

  @Test
  void rejectsNegativeAmounts() {
    var error = assertThrows(IllegalArgumentException.class,
        () -> Money.of("-1"));
    assertEquals("negative amount: -1", error.getMessage());
  }

  // addsAndMultiplies() is shown on p.08
}
```

The class under test is a small `Money` record (₹, two decimals). Test classes and methods can be package-private; JUnit finds any method annotated with `@Test`.

The annotations you'll use daily

`@Test`	A test method. No return value, not private.
`@ParameterizedTest`	Runs once per set of arguments (see p.09).
`@BeforeEach` / `@AfterEach`	Run around **every** test: fresh fixtures, clean-up.
`@BeforeAll` / `@AfterAll`	Once per class; `static` unless `@TestInstance(PER_CLASS)`.
`@DisplayName`	Readable name in reports, any text or emoji.
`@Nested`	Inner class that groups related tests and shares setup.
`@Tag("slow")`	Label for filtering what runs, e.g. in CI stages.
`@Disabled("why")`	Skip, with a reason. Prefer fixing or deleting.
`@TempDir`	Inject a fresh temporary directory, deleted afterwards.
`@ExtendWith`	Plug in an extension, e.g. `MockitoExtension`.

Lifecycle, observed

LifecycleTest.java

```java
class LifecycleTest {
  static void log(String s) { System.out.println(s); }

  LifecycleTest() { log("  new LifecycleTest()"); }

  @BeforeAll static void beforeAll() { log("@BeforeAll"); }
  @BeforeEach void beforeEach()      { log("  @BeforeEach"); }

  @Test void first()                 { log("    first()"); }
  @Test void second()                { log("    second()"); }

  @AfterEach void afterEach()        { log("  @AfterEach"); }
  @AfterAll static void afterAll()   { log("@AfterAll"); }
}
```

real output

```
@BeforeAll
  new LifecycleTest()
  @BeforeEach
    second()
  @AfterEach
  new LifecycleTest()
  @BeforeEach
    first()
  @AfterEach
@AfterAll
```

- **A new instance per test method**, so fields can't leak state between tests.
- **Order is deterministic but deliberately not obvious** — here `second()` ran first. Never depend on it.
- Need an order? `@TestMethodOrder(OrderAnnotation.class)` — mostly for slow, staged integration tests.
- `@TestInstance(PER_CLASS)` reuses one instance, so `@BeforeAll` need not be static.

What changed in JUnit 6

CHANGE	WHAT IT MEANS FOR YOU
Java 17 minimum	Projects on older JDKs stay on JUnit 5.14.x until they upgrade.
One version for everything	Platform, Jupiter and Vintage now share one number, so the BOM is simpler.
Vintage deprecated	JUnit 4 tests still run, but the engine flags them at discovery; plan the migration.
Quoted text arguments	Parameterized names show `"1000"`; turn off with `quoteTextArguments = false`.
CSV parsing via FastCSV	`@CsvSource` is stricter: characters after a closing quote are now an error.

02 · UNIT TESTING WITH JUNIT 6

Assertions & AssertJ

An assertion is the test's oracle. JUnit's `Assertions` class covers the basics; **AssertJ** adds a fluent style — type `assertThat(x).` and your IDE lists every check that makes sense for `x` — and much clearer failure messages.

JUnit assertions

`assertEquals(expected, actual)`	Uses `equals()`. Expected first!
`assertTrue / assertFalse`	For booleans; message explains little.
`assertNull / assertNotNull`	Null checks.
`assertSame`	Same object reference (`==`).
`assertIterableEquals`	Same elements, same order.
`assertThrows(Type, code)`	Returns the exception for further checks.
`assertThrowsExactly`	Exact type, not a subclass.
`assertDoesNotThrow`	Documents that code must not fail.
`assertAll(…)`	Runs every check and reports all failures.
`assertTimeout(Preemptively)`	Fails when slower than a limit.
`assertInstanceOf`	Type check that returns the cast value.

Exceptions and grouped checks

MoneyTest.java — ✔ both pass

```
@Test
void rejectsNegativeAmounts() {
  var error = assertThrows(IllegalArgumentException.class,
      () → Money.of("-1"));
  assertEquals("negative amount: -1", error.getMessage());
}

@Test
void addsAndMultiplies() {
  Money m = Money.of("12.50");
  assertAll("arithmetic",
      () → assertEquals(Money.of("25.00"), m.plus(m)),
      () → assertEquals(Money.of("37.50"), m.times(3)),
      () → assertEquals("₹12.50", m.toString()));
}
```

The same ideas in AssertJ

ItemTest.java

```
void rejected(int qty) {
  assertThatThrownBy(() → new Item("SKU-1", Money.of("10"), qty))
    .isInstanceOf(IllegalArgumentException.class)
    .hasMessageContaining("quantity");
}
```

Failure messages are the real product

A test earns its keep the day it fails. These are the exact messages JUnit and AssertJ printed when we broke an expectation on purpose:

assertEquals

```
assertEquals(Money.of("900.00"),
    Money.of("999.99").percentOff(10));
```

JUnit says

```
expected: <₹900.00> but was: <₹899.99>
```

assertAll

```
assertAll("arithmetic",
  () → assertEquals(Money.of("25.00"), m.times(3)),
  () → assertEquals("Rs 12.50", m.toString()));
```

JUnit says — both failures

```
arithmetic (2 failures)
  AssertionFailedError: expected: <₹25.00> but was: <₹37.50>
  AssertionFailedError: expected: <Rs 12.50> but was: <₹12.50>
```

AssertJ

```
assertThat(List.of("PEN", "BOOK", "TV"))
    .containsExactly("PEN", "TV", "BOOK");
```

AssertJ says

```
Expecting actual:
  ["PEN", "BOOK", "TV"]
to contain exactly (and in same order):
  ["PEN", "TV", "BOOK"]
but there were differences at these indexes:
  - element at index 1: expected "TV" but was "BOOK"
  - element at index 2: expected "BOOK" but was "TV"
```

WATCH OUT

Put **expected first** in `assertEquals(expected, actual)`. Swapped arguments still fail correctly, but the message lies about which value was wrong. AssertJ avoids the problem: `assertThat(actual).isEqualTo(expected)`.

TIP

Compare **money with** `BigDecimal` and a fixed scale. `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is `false`; that's why `Money` always calls `setScale(2)`.

02 · UNIT TESTING WITH JUNIT 6

Parameterized, nested & more

When several tests differ only in their data, write one parameterized test. Each row becomes its own entry in the report, so you see exactly which case failed.

ItemTest.java — @Nested + @MethodSource + @ValueSource

```java
@DisplayName("Item")
class ItemTest {

  @Nested @DisplayName("when quantity is valid")
  class ValidQuantity {
    @ParameterizedTest(name = "{0} × ₹{1} = ₹{2}",
            quoteTextArguments = false)
    @MethodSource("shop.ItemTest#totals")
    void totalIsPriceTimesQty(int qty, String price, String total) {
      assertThat(new Item("SKU-1", Money.of(price), qty).total())
        .isEqualTo(Money.of(total));
    }
  }

  @Nested @DisplayName("when quantity is invalid")
  class InvalidQuantity {
    @ParameterizedTest(name = "quantity {0} is rejected")
    @ValueSource(ints = {0, -1, Integer.MIN_VALUE})
    void rejected(int qty) {
      assertThatThrownBy(() -> new Item("SKU-1", Money.of("10"), qty))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("quantity");
    }
  }

  static Stream<Arguments> totals() {
    return Stream.of(
      Arguments.of(1, "10", "10"),
      Arguments.of(3, "19.99", "59.97"),
      Arguments.of(1000, "0.01", "10"));
  }
}
```

real output 6 / 6 passed

```
Item ✔
├─ when quantity is invalid ✔
│  └─ rejected(int) ✔
│     ├─ quantity 0 is rejected ✔
│     ├─ quantity -1 is rejected ✔
│     └─ quantity -2147483648 is rejected ✔
└─ when quantity is valid ✔
   └─ totalIsPriceTimesQty(int, String, String) ✔
      ├─ 1 × ₹10 = ₹10 ✔
      ├─ 3 × ₹19.99 = ₹59.97 ✔
      └─ 1000 × ₹0.01 = ₹10 ✔
```

JUNIT 6 CHANGE

JUnit 6 prints text arguments in quotes, and `@CsvSource` values count as text — even numbers. `quoteTextArguments = false` keeps names like **₹1000 → 5% off**.

RULE OF THUMB

Keep each row **one behaviour**. If rows need different assertions, they are different tests.

Argument sources

`@ValueSource(ints = {0, -1})`	One literal argument per run
`@NullAndEmptySource`	Adds `null` and `""` /empty collection
`@EnumSource(Status.class)`	Every enum constant, or a filtered set
`@CsvSource({"a, 1"})`	Several arguments per row, inline
`@CsvFileSource("/cases.csv")`	Rows from a CSV file on the classpath
`@MethodSource("cases")`	A static factory returning `Stream<Arguments>`
`@FieldSource("cases")`	A static field holding the arguments (JUnit 5.11+)
`@ArgumentsSource(My.class)`	Your own reusable `ArgumentsProvider`

More tools in the box

Tags & filtering

`@Tag("slow")` on a class or method, then `mvn test -Dgroups=fast` or `-DexcludedGroups=slow`.

Assumptions

`assumeTrue(isCi())` aborts (skips) the test instead of failing it when a precondition doesn't hold.

Conditional tests

`@EnabledOnOs(LINUX)`, `@EnabledForJreRange(min = JAVA_21)`, `@EnabledIfEnvironmentVariable`.

Timeouts

`@Timeout(5)` on a method or class; `assertTimeoutPreemptively` for one block (see p.17).

Temp files

`@TempDir Path dir` gives each test a fresh folder and deletes it afterwards.

Extensions

`@ExtendWith` / `@RegisterExtension` hook into the lifecycle: Mockito, Spring, Testcontainers all work this way.

CHAPTER THREE

03 Test doubles & Mockito

A unit test shouldn't send real e-mails or charge real cards. A **test double** stands in for a collaborator, like a stunt double for an actor. The five kinds come from Gerard Meszaros's *xUnit Test Patterns* (2007).

The five test doubles

DOUBLE	WHAT IT IS	WHAT YOU VERIFY	IN THIS GUIDE
`dummy`	Fills a parameter list; never actually used.	Nothing	`dummyEmail` throws if touched
`stub`	Returns canned answers to calls.	State of the SUT	`alwaysApproves` payment gateway
`spy`	A stub that also records the calls it got.	Behaviour, checked by you	`SpyEmailSender.sentTo`
`mock`	Pre-programmed with expectations; the library checks them.	Behaviour	Mockito `verify(email).send(…)`
`fake`	A working, simplified implementation.	State, realistically	`InMemoryOrderRepository`

TestDoublesTest.java — no library needed ✔ 3 / 3 passed

```java
/** The five kinds of test double, hand-written — no library needed. */
class TestDoublesTest {

  // DUMMY: passed in to satisfy a parameter, never used
  EmailSender dummyEmail = (to, subject, body) -> {
    throw new AssertionError("not used");
  };

  // STUB: returns canned answers
  PaymentGateway alwaysApproves = (customer, amount) -> "PAY-1";

  // SPY: a stub that also records how it was called
  static class SpyEmailSender implements EmailSender {
    final List<String> sentTo = new ArrayList<>();
    @Override public void send(String to, String subject, String body) {
      sentTo.add(to);
    }
  }

  // FAKE: a real but simplified implementation (a map instead of a DB)
  OrderRepository fakeRepo = new InMemoryOrderRepository();

  List<Item> pens = List.of(new Item("PEN", Money.of("50"), 2));

  OrderService serviceWith(EmailSender email) {
    return new OrderService(fakeRepo, alwaysApproves, email,
        new DiscountPolicy(), Clock.systemUTC());
  }

  @Test
  void stubAndFakeLetUsCheckState() {
    var service = serviceWith(new SpyEmailSender());
    Order order = service.placeOrder("meera@example.com", pens);
    assertThat(fakeRepo.findById(order.id())).contains(order);  // state
  }

  @Test
  void spyLetsUsCheckBehaviour() {
    var spy = new SpyEmailSender();
    serviceWith(spy).placeOrder("meera@example.com", pens);
    assertThat(spy.sentTo)
        .containsExactly("meera@example.com");  // behaviour
  }

  @Test
  void dummyIsNeverTouchedWhenValidationFails() {
    var service = serviceWith(dummyEmail);
    assertThatIllegalArgumentException()
      .isThrownBy(() -> service.placeOrder("x@example.com", List.of()));
  }
}
```

State or behaviour?

State verification

Run the code, then look at the result: what was returned, or what is now stored in the fake. Survives refactoring well, because it doesn't care *how* the result was reached.

Behaviour verification

Check which calls were made to collaborators: "an e-mail was sent to Meera". Needed when the call *is* the result — sending, publishing, charging — and nothing observable is stored.

03 · TEST DOUBLES & MOCKITO

Mockito in practice

`placeOrder` applies the discount, charges the card, saves the order and e-mails the customer. In the test the gateway, repository and mailer are mocks; the clock is fixed.

OrderService.java — under test

```java
public Order placeOrder(String customerEmail, List<Item> items) {
  if (items.isEmpty())
    throw new IllegalArgumentException("order has no items");
  Money subtotal = items.stream().map(Item::total)
      .reduce(Money.of("0"), Money::plus);
  Money total = subtotal.percentOff(discounts.percentFor(subtotal));
  payments.charge(customerEmail, total);  // may throw PaymentDeclined…
  Order order = repository.save(new Order(UUID.randomUUID().toString(),
      customerEmail, List.copyOf(items), total, clock.instant()));
  email.send(customerEmail, "Order confirmed", "Total: " + total);
  return order;
}
```

- Every collaborator arrives through the **constructor**, so a test can pass in doubles — no framework needed.
- Time comes from an injected `java.time.Clock`, so the timestamp is predictable.
- `placeOrder` has one happy path and two failure paths: no items, or a declined card.

OrderServiceTest.java (first test)

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

  @Mock OrderRepository repository;
  @Mock PaymentGateway payments;
  @Mock EmailSender email;
  @Captor ArgumentCaptor<Order> savedOrder;

  Clock clock = Clock.fixed(Instant.parse("2026-10-13T10:00:00Z"),
                            ZoneOffset.UTC);
  OrderService service;

  @BeforeEach
  void setUp() {
    service = new OrderService(repository, payments, email,
        new DiscountPolicy(), clock);
  }

  @Test
  void chargesDiscountedTotalSavesAndEmails() {
    when(repository.save(any())).thenAnswer(c -> c.getArgument(0));

    Order order = service.placeOrder("asha@example.com",
        List.of(new Item("BOOK", Money.of("600"), 2))); // ₹1200 → 5%

    assertThat(order.total()).isEqualTo(Money.of("1140"));
    verify(payments).charge("asha@example.com", Money.of("1140"));
    verify(repository).save(savedOrder.capture());
    assertThat(savedOrder.getValue().placedAt())
        .isEqualTo(clock.instant());
    verify(email).send(eq("asha@example.com"), eq("Order confirmed"),
        contains("₹1140.00"));
  }
```

real output — 4 / 4 passed

```
OrderServiceTest ✔
├─ chargesBeforeSaving() ✔
├─ chargesDiscountedTotalSavesAndEmails() ✔
├─ declinedPaymentSavesNothingAndSendsNoEmail() ✔
└─ emptyOrderIsRejectedEarly() ✔
```

- **@ExtendWith(MockitoExtension.class)** creates the `@Mock` and `@Captor` fields before each test and checks for unused stubs after it.
- **thenAnswer** makes `save` return its argument, like a real repository.
- **ArgumentCaptor** grabs the saved `Order` so we can check its timestamp came from the fixed clock.
- **eq** and **contains** are matchers: one matcher means all arguments need one.

Constructor beats @InjectMocks

`@InjectMocks` builds the object by constructor, setter or field injection — and fails *silently* if it can't. Calling the constructor yourself is explicit and breaks loudly.

Defaults of a mock

An unstubbed mock returns empty values: `null`, `0`, `false`, empty collections and `Optional.empty()`. Void methods simply do nothing.

Stub less, verify less

Stub only what the code path needs and verify only calls that are the point of the test — the rest is noise that breaks on refactors.

Classicist *vs* Mockist

Classicist — Use real objects where possible, doubles only for awkward ones (network, time, mail).

Mockist — Replace every collaborator with a mock and verify the conversation between objects.

From Fowler's *Mocks Aren't Stubs* (2007). Most teams lean classicist.

GOOD TO KNOW

Mockito's `mock()` can be a stub, spy or mock depending on use — the library blurs Meszaros's names. The five kinds still help you decide what to check.

03 · TEST DOUBLES & MOCKITO

Failure paths, call order & the API

OrderServiceTest.java — failure path and call order

```java
@Test
void declinedPaymentSavesNothingAndSendsNoEmail() {
  when(payments.charge(anyString(), any()))
    .thenThrow(new PaymentDeclinedException("card expired"));

  assertThatThrownBy(() -> service.placeOrder("asha@example.com",
      List.of(new Item("PEN", Money.of("50"), 1))))
    .isInstanceOf(PaymentDeclinedException.class)
    .hasMessage("card expired");

  verifyNoInteractions(repository, email);
}

@Test
void chargesBeforeSaving() {
  when(repository.save(any())).thenAnswer(c -> c.getArgument(0));

  service.placeOrder("asha@example.com",
      List.of(new Item("PEN", Money.of("50"), 1)));

  InOrder inOrder = inOrder(payments, repository, email);
  inOrder.verify(payments).charge(any(), any());
  inOrder.verify(repository).save(any());
  inOrder.verify(email).send(any(), any(), any());
}
```

The API on one table

Create	`mock(X.class)` · `@Mock` · `spy(obj)` · `@Spy` · `@Captor` · `@InjectMocks`
Stub	`when(m.call()).thenReturn(v)` · `.thenThrow(e)` · `.thenAnswer(inv → …)`
Stub void / spies	`doThrow(e).when(m).call()` · `doReturn(v).when(spy).call()` · `doNothing()`
Verify	`verify(m).call()` · `times(2)` · `never()` · `atLeastOnce()` · `timeout(100)` · `verifyNoInteractions(m)`
Order	`InOrder o = inOrder(a, b);` `o.verify(a).x(); o.verify(b).y();`
Match	`any()` · `anyString()` · `eq(v)` · `contains(s)` · `argThat(x → …)` · `isNull()`
Capture	`verify(m).save(captor.capture());` `captor.getValue()`
BDD style	`given(m.call()).willReturn(v)` · `then(m).should().call()`

Given / when / then with BDDMockito

OrderServiceBddTest.java — ✔ passes

```java
  @Test
  void bigOrderGetsTenPercentOff() {
    // given
    given(repository.save(any())).willAnswer(c -> c.getArgument(0));
    var service = new OrderService(repository, payments, email,
        new DiscountPolicy(), Clock.systemUTC());

    // when
    Order order = service.placeOrder("ravi@example.com",
        List.of(new Item("TV", Money.of("5000"), 1)));

    // then
    assertThat(order.total()).isEqualTo(Money.of("4500"));
    then(payments).should().charge("ravi@example.com", Money.of("4500"));
    then(email).should(times(1)).send(any(), any(), any());
  }
}
```

`BDDMockito` renames the same calls so tests read like a specification: `given(…).willReturn(…)` instead of `when(…).thenReturn(…)`, and `then(mock).should()` instead of `verify(mock)`. Pick one style per code base: mixing them makes readers wonder whether they behave differently — they don't.

Rules that keep mocks honest

Don't mock what you don't own

Wrap a third-party client (Stripe, S3, an HTTP SDK) in your own small interface and mock *that*. Test the wrapper against the real thing in an integration test.

Don't mock values

Records, `Money`, `List`, `LocalDate`: just create real ones. Mocking them makes tests longer and hides bugs.

Matchers: all or nothing

If one argument uses a matcher, every argument must: `eq("a"), any()`, not `"a", any()` — Mockito throws `InvalidUseOfMatchersException`.

A mock returning a mock is a smell

`when(a.getB().getC())` means the code reaches through objects. Pass what it needs directly.

Verify outcomes, not every call

`verifyNoMoreInteractions` on every test makes harmless refactors fail. Verify the calls that matter.

Too many mocks

Five or more mocks in one test usually means the class does too much. Split it, or test a bigger unit with fakes.

03 · TEST DOUBLES & MOCKITO

Spies, statics, errors & pitfalls

MockitoFeaturesTest.java ✔ 4 / 4 passed

```java
class MockitoFeaturesTest {

  @Test
  void consecutiveAnswersForRetries() {
    PaymentGateway gateway = mock(PaymentGateway.class);
    when(gateway.charge(any(), any()))
      .thenThrow(new PaymentDeclinedException("timeout"))
      .thenReturn("PAY-2");

    Money one = Money.of("1");
    assertThatThrownBy(() -> gateway.charge("a@example.com", one));
    assertThat(gateway.charge("a@example.com", one)).isEqualTo("PAY-2");
    verify(gateway, times(2)).charge(any(), any());
  }

  @Test
  void spyWrapsARealObject() {
    List<String> list = spy(new ArrayList<>());
    list.add("one");
    // on spies use doReturn: when(list.size()) calls the real method
    doReturn(100).when(list).size();

    assertThat(list.size()).isEqualTo(100);
    assertThat(list.get(0)).isEqualTo("one");  // real behaviour kept
    verify(list).add("one");
  }

  @Test
  void mockStaticIsScopedToTheTryBlock() {
    UUID fixed = UUID.fromString("0-0-0-0-42");
    try (MockedStatic<UUID> uuid =
             mockStatic(UUID.class, CALLS_REAL_METHODS)) {
      uuid.when(UUID::randomUUID).thenReturn(fixed);
      assertThat(UUID.randomUUID()).isEqualTo(fixed);
    }
    assertThat(UUID.randomUUID()).isNotEqualTo(fixed); // real again
  }

  @Test
  void finalClassesCanBeMockedByDefault() {
    DiscountPolicy policy = mock(DiscountPolicy.class); // final class
    when(policy.percentFor(any())).thenReturn(50);
    assertThat(policy.percentFor(Money.of("10"))).isEqualTo(50);
  }
}
```

Reading Mockito's errors

verify(payments).charge(any(), any());

```
Wanted but not invoked:
payments.charge(<any>, <any>);
→ at
shop.FailingExamplesTest.f_wantedButNotInvoked(Fail
ingExamplesTest.java:45)
Actually, there were zero interactions with this
mock.
```

verify(email).send(…, "Total: ₹1200.00");

```
Argument(s) are different! Wanted:
email.send(
    "asha@example.com",
    "Order confirmed",
    "Total: ₹1200.00"
);
→ at
shop.FailingExamplesTest.e_argumentMismatch(Failing
ExamplesTest.java:41)
Actual invocations have different arguments at
position [2]:
email.send(
    "asha@example.com",
    "Order confirmed",
    "Total: ₹1140.00"
);
→ at
shop.FailingExamplesTest.e_argumentMismatch(Failing
ExamplesTest.java:40)
```

a when(…) the test never used

```
Unnecessary stubbings detected.
Clean & maintainable test code requires zero
unnecessary code.
Following stubbings are unnecessary (click to
navigate to relevant line of code):
1. → at
shop.FailingExamplesTest.d_unnecessaryStubbing(Fail
ingExamplesTest.java:35)
Please remove unnecessary stubbings or use
'lenient' strictness. More info: javadoc for
UnnecessaryStubbingException class.
```

Strictness: `MockitoExtension` defaults to `STRICT_STUBS` — unused stubs fail the test. `WARN` only logs; `LENIENT` ignores them. Change it with `@MockitoSettings(strictness = …)` or per stub with `lenient().when(…)`. An unused `when(…)` is dead setup that misleads readers, so keep the default. The three messages above are real output from deliberately failing tests.

JDK 21+ WARNING

Mockito 5's default **inline mock maker** (final classes, `mockStatic`) attaches a Java agent at run time. JDK 21+ warns about that (JEP 451) and a future JDK may refuse it — load Mockito as an agent in the build instead. `@{{argLine}}` must already be defined (JaCoCo's `prepare-agent` sets it; otherwise declare an empty `argLine` property).

pom.xml — Mockito as a -javaagent (from Mockito's docs)

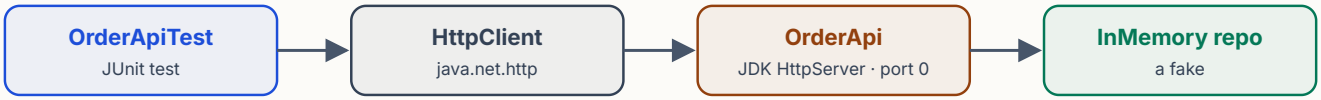
```xml
<plugin>  <!-- defines ${org.mockito:mockito-core:jar} -->
  <artifactId>maven-dependency-plugin</artifactId>
  <executions>
    <execution><goals><goal>properties</goal></goals></execution>
  </executions>
</plugin>
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <argLine>@{argLine} -javaagent:${org.mockito:mockito-core:jar}</argLine>
  </configuration>
</plugin>
```

CHAPTER FOUR

04 Integration & API testing

Unit tests prove each class works alone; integration tests prove they work *together* — with real HTTP, serialisation, SQL and configuration. They are slower, so keep them focused.

Testing a real HTTP API



OrderApiTest.java

```java
/** Integration test: a real HTTP server on a free port + a real client. */
class OrderApiTest {
  static final OrderRepository repo = new InMemoryOrderRepository();
  static final OrderApi api = new OrderApi(repo);
  static final HttpClient http = HttpClient.newHttpClient();
  static String baseUrl;

  @BeforeAll static void startServer() throws Exception {
    baseUrl = "http://127.0.0.1:" + api.start();
    repo.save(new Order("ord-1", "asha@example.com",
        List.of(new Item("BOOK", Money.of("600"), 2)),
        Money.of("1140"), Instant.EPOCH));
  }

  @AfterAll static void stopServer() { api.stop(); }

  HttpResponse<String> send(String method, String path)
      throws Exception {
    var request = HttpRequest.newBuilder(URI.create(baseUrl + path))
        .method(method, HttpRequest.BodyPublishers.noBody()).build();
    return http.send(request, HttpResponse.BodyHandlers.ofString());
  }

  @Test void existingOrderReturns200WithJson() throws Exception {
    var res = send("GET", "/orders/ord-1");
    assertThat(res.statusCode()).isEqualTo(200);
    assertThat(res.headers().firstValue("Content-Type"))
      .hasValue("application/json");
    assertThat(res.body())
      .isEqualTo("{\"id\":\"ord-1\",\"total\":\"1140.00\",\"items\":1}");
  }

  @Test void unknownOrderReturns404() throws Exception {
    assertThat(send("GET", "/orders/nope").statusCode()).isEqualTo(404);
  }

  @Test void wrongMethodReturns405WithAllowHeader() throws Exception {
    var res = send("DELETE", "/orders/ord-1");
    assertThat(res.statusCode()).isEqualTo(405);
    assertThat(res.headers().firstValue("Allow")).hasValue("GET");
  }

  // pathTricksAreJustNotFound() → p.20
}
```

real output — 4 / 4 passed

```
OrderApiTest ✔
├─ wrongMethodReturns405WithAllowHeader() ✔
├─ unknownOrderReturns404() ✔
├─ pathTricksAreJustNotFound() ✔
└─ existingOrderReturns200WithJson() ✔
```

- **Port 0** lets the OS pick a free port, so tests never clash with each other or with a running app.
- **@BeforeAll / @AfterAll** start the server once per class — starting it per test would be slow.
- Check the **status, headers and body**: a 200 with the wrong `Content-Type` is still a bug.
- Test the **unhappy paths** too: 404 for a missing order, 405 plus an `Allow` header for a wrong method.
- JDK `HttpServer` + `HttpClient` keep it dependency-free; Spring Boot's `RANDOM_PORT` gives the same shape.

Narrow or broad?

Narrow integration test vs **Broad integration test**

Exercises *one* boundary — repository vs real database, client vs stubbed HTTP server.

Starts most of the application and checks a whole flow through several boundaries.

Prefer many narrow tests: their failures point straight at the broken boundary.

In-memory stand-in vs **The real thing in a container**

H2 for PostgreSQL, a map for Redis. Fast, but dialects and locking differ.

Testcontainers runs production's PostgreSQL version in Docker.

Fakes for *unit* tests; the real engine when the database is what you're testing.

04 · INTEGRATION & API TESTING

Spring Boot, Testcontainers & contracts

In a Spring Boot application, a **test slice** starts only the part of the context one layer needs, so it is far faster than `@SpringBootTest`. Spring Boot 4.0 (November 2025) moved each slice into its own test module.

ANNOTATION	LOADS	BOOT 4 MODULE
`@WebMvcTest`	Controllers, filters, JSON, validation; no services or DB	`spring-boot-webmvc-test`
`@WebFluxTest`	The same for reactive WebFlux controllers	`spring-boot-webflux-test`
`@DataJpaTest`	JPA repositories, entities, transactions (rolled back)	`spring-boot-data-jpa-test`
`@JdbcTest`	`JdbcTemplate` / `JdbcClient` code	`spring-boot-jdbc-test`
`@JsonTest`	Jackson / Gson (de)serialisation only	`spring-boot-test-autoconfigure`
`@RestClientTest`	Your HTTP clients against a mock server	`spring-boot-restclient-test`
`@SpringBootTest`	The whole application context — the broad test	`spring-boot-test`

OrderControllerTest — controller slice

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
  @Autowired MockMvcTester mvc;       // AssertJ-style MockMvc
  @MockitoBean OrderService orders;   // replaces the real bean

  @Test
  void knownOrderIsReturnedAsJson() {
    given(orders.find("A-1")).willReturn(Optional.of(
        new OrderView("A-1", new BigDecimal("1140.00"), 2)));

    assertThat(mvc.get().uri("/orders/A-1"))
        .hasStatusOk()
        .bodyJson().isLenientlyEqualTo("""
            {"reference":"A-1","total":1140.00,"lines":2}""");
  }

  @Test
  void unknownOrderIs404() {
    given(orders.find("nope")).willReturn(Optional.empty());

    assertThat(mvc.get().uri("/orders/nope"))
        .hasStatus(HttpStatus.NOT_FOUND);
  }
}
```

TestcontainersConfiguration — PostgreSQL as a bean

```
@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {
  @Bean
  @ServiceConnection                  // Boot wires the JDBC URL
  PostgreSQLContainer postgres() {
    return new PostgreSQLContainer("postgres:17-alpine");
  }
}
```

real run · Spring Boot 4.1.1 on GitHub Actions

```
✔ OrderControllerTest  2 tests  3.099 s
✔ OrderRepositoryIT    2 tests  14.73 s
✔ OrderFlowIT          3 tests  3.866 s
```

Versions managed by Boot 4.1.1: Spring Framework 7.0.9, JUnit 6.0.3, Mockito 5.23.0, Hibernate 7.4.5.Final, Testcontainers 2.0.5. Project: `spring/` in the example code. Our first version used `static @Container` fields per class; that run's test JVM took 30 s to exit and Failsafe had to kill it. With the container as a Spring bean it closes with the context, and the JVM exited at once.

@MockitoBean, not @MockBean

Spring Framework 6.2 added `@MockitoBean` and `@MockitoSpyBean`. Boot 3.4 deprecated `@MockBean` / `@SpyBean`, and Boot 4.0 removed them.

@ServiceConnection

Since Boot 3.1: Boot sees the container's type and wires the connection details for you — no `spring.datasource.url` juggling.

Testcontainers 2.0

October 2025. Module artifacts gained a prefix: `testcontainers-postgresql`, `testcontainers-junit-jupiter`.

Calling other services: stubs and contracts

WireMock runs a real HTTP server that returns canned responses, so your client code sends real requests over a socket. Stub the slow, the broken and the odd: timeouts, 500s, malformed JSON, redirects.

The risk: your stub and the real provider drift apart, and both sides stay green while production breaks.

Consumer-driven contract tests (Ian Robinson, 2006) close that gap. The consumer records what it needs from the API; the provider's build replays those expectations against the real service. Tools: **Pact** (many languages, a broker to share contracts) and **Spring Cloud Contract** (JVM, contracts generate stubs and provider tests).

04 · INTEGRATION & API TESTING

The `@Transactional` test trap

`@DataJpaTest` — and any test class annotated `@Transactional` — runs each test inside a transaction that is **rolled back**. That keeps the database clean, but nothing is ever committed, so two kinds of production bug can't happen in the test. The same code, run both ways against real PostgreSQL:

rolled back · OrderRepositoryIT — 2 ✔

```
@DataJpaTest
@Import(TestcontainersConfiguration.class)
class OrderRepositoryIT {
  @Autowired OrderRepository repository;

  @Test
  void duplicateReferenceSlipsThroughUntilFlush() {
    repository.save(new CustomerOrder("A-1", BigDecimal.ONE));
    repository.save(new CustomerOrder("A-1", BigDecimal.TWO));
    // no error yet: both INSERTs wait in the session

    assertThatThrownBy(repository::flush)
        .isInstanceOf(DataIntegrityViolationException.class);
  }

  @Test
  void lazyLinesLoadBecauseTheTestHoldsATransaction() {
    repository.saveAndFlush(
        new CustomerOrder("B-1", BigDecimal.TEN)
            .addLine("BOOK", 2).addLine("PEN", 1));

    var order = repository.findByReference("B-1").orElseThrow();
    assertThat(order.getLines()).hasSize(2);   // green here …
  }
}
```

committed · OrderFlowIT (@SpringBootTest, no @Transactional) — 3 ✔

```
@Autowired OrderRepository repository;

@BeforeEach
void cleanSlate() { repository.deleteAll(); } // no rollback

@Test
void duplicateReferenceFailsOnSave() {
  repository.save(new CustomerOrder("A-1", BigDecimal.ONE));

  assertThatThrownBy(() -> repository.save(
          new CustomerOrder("A-1", BigDecimal.TWO)))
      .isInstanceOf(DataIntegrityViolationException.class);
}

@Test
void lazyLinesFailOutsideATransaction() {
  repository.save(
      new CustomerOrder("B-1", BigDecimal.TEN)
          .addLine("BOOK", 2).addLine("PEN", 1));

  var order = repository.findByReference("B-1").orElseThrow();
  assertThatThrownBy(() -> order.getLines().size()) // red here
      .isInstanceOf(LazyInitializationException.class);
}
```

Same code, two outcomes

	INSIDE A ROLLED-BACK TEST TRANSACTION	COMMITTED, AS IN PRODUCTION
Duplicate `reference`	`save()` succeeds; the error appears only at `flush()` — and a test that never flushes stays green	`save()` commits and throws `DataIntegrityViolationException`
Lazy `lines` collection	`getLines()` loads, because the test's transaction is still open	`LazyInitializationException` — the real behaviour with `open-in-view=false`
Data left behind	none — rolled back after each test	rows stay: clean up in `@BeforeEach`

PostgreSQL's message in both runs: `ERROR: duplicate key value violates unique constraint "orders_reference_key"`. Spring translates it to `DataIntegrityViolationException`. Every test above passes — each one asserts what really happens.

The fix the committed test protects

OrderService.java — map to plain data inside the transaction

```
/** Map to a view while the transaction is still open. */
@Transactional(readOnly = true)
public Optional<OrderView> find(String reference) {
  return repository.findWithLines(reference).map(OrderView::of);
}
```

- **No `@Transactional` on `@SpringBootTest` tests.** Let each call commit as it would in production; clean up in `@BeforeEach`.
- **In `@DataJpaTest`, flush.** Testing a constraint or a mapping? Call `flush()` or `saveAndFlush()` so the SQL actually runs.
- **Turn off open-in-view** (`spring.jpa.open-in-view=false`) so lazy loading can't silently happen while JSON is written.
- **Return DTOs, not entities,** from transactional services — then no lazy proxy can escape.
- Keep the fast rolled-back tests; add **one committed-path test** per repository or flow.

real run · Spring Boot 4.1.1 · GitHub Actions

```
✔ OrderRepositoryIT  2 tests  14.73 s  (starts PostgreSQL)
✔ OrderFlowIT        3 tests  3.866 s

WARN  org.hibernate.orm.jdbc.error : ERROR: duplicate key value
violates unique constraint "orders_reference_key"
```

GOOD TO KNOW

Rolled-back tests are still the right default for queries, mappings and repository methods — they are fast and leave nothing behind. Call `flush()` and `clear()` on the `EntityManager` (or `TestEntityManager`) before reading back, so the data really comes from the database.

04 · INTEGRATION & API TESTING

Time, randomness, threads & flaky tests

A test that passes and fails without any code change is **flaky**. Flaky tests are worse than none: people learn to press "re-run", and real failures get ignored. Almost every cause is something the test doesn't control.

TrickyThingsTest.java

```java
class TrickyThingsTest {

  @Test void timeComesFromAnInjectedClock() {
    Instant newYearsEve = Instant.parse("2026-12-31T23:59:59Z");
    Clock clock = Clock.fixed(newYearsEve, ZoneOffset.UTC);
    var service = new OrderService(new InMemoryOrderRepository(),
        (c, m) -> "PAY", (to, s, b) -> {}, new DiscountPolicy(), clock);

    Order o = service.placeOrder("a@example.com",
        List.of(new Item("PEN", Money.of("1"), 1)));

    assertThat(o.placedAt()).isEqualTo(newYearsEve);
  }

  @Test void randomnessComesFromASeed() {
    var a = new Random(42).ints(5, 0, 100).boxed().toList();
    var b = new Random(42).ints(5, 0, 100).boxed().toList();
    assertThat(a).isEqualTo(b);  // same seed, same sequence
  }

  @Test void filesGoInAFreshTempDir(@TempDir Path dir) throws Exception {
    Path report = dir.resolve("report.csv");
    Files.writeString(report, "sku,qty\nPEN,2\n");
    assertThat(Files.readAllLines(report))
        .containsExactly("sku,qty", "PEN,2");
  }  // JUnit deletes dir afterwards

  @Test void concurrentSavesAreAllKept() throws Exception {
    var repo = new InMemoryOrderRepository();
    int threads = 16, perThread = 500;
    var start = new CountDownLatch(1);
    try (ExecutorService pool = Executors.newFixedThreadPool(threads)) {
      for (int t = 0; t < threads; t++) {
        int base = t * perThread;
        pool.submit(() -> {
          start.await();  // release all threads at once
          for (int i = 0; i < perThread; i++)
            repo.save(new Order("o" + (base + i), "x@example.com",
                List.of(), Money.of("0"), Instant.EPOCH));
          return null;
        });
      }
      start.countDown();
    }  // close() waits for all tasks
    for (int i = 0; i < threads * perThread; i++)
      assertThat(repo.findById("o" + i)).isPresent();
  }

  @Test void slowCodeFailsFastInsteadOfHangingTheBuild() {
    assertTimeoutPreemptively(Duration.ofSeconds(1),
        () -> new DiscountPolicy().percentFor(Money.of("1")));
  }
}
```

real output — 5 / 5 passed

```
TrickyThingsTest ✔
├─ randomnessComesFromASeed() ✔
├─ timeComesFromAnInjectedClock() ✔
├─ filesGoInAFreshTempDir(Path) ✔
├─ concurrentSavesAreAllKept() ✔
└─ slowCodeFailsFastInsteadOfHangingTheBuild() ✔
```

Where flakiness comes from

CAUSE	FIX
Real time `LocalDate.now()` breaks at midnight, month end, DST	Inject a `Clock`; use `Clock.fixed`
Randomness UUIDs, shuffles, random test data	Seed it, or inject the generator
Sleeping `Thread.sleep(500)` is too short on a busy CI box	Wait for a condition (Awaitility) with a timeout
Shared state Static fields, a shared DB row, a singleton cache	Fresh fixtures per test; unique IDs; reset in `@AfterEach`
Test order Test B only passes after test A	Each test sets up its own data; run in random order to find these
Time zone / locale Passes in Bengaluru, fails on a UTC build agent	Set zone and locale explicitly in code and tests
Network Calls to real external services	Stub at the boundary; containers for owned dependencies
Concurrency Races appear one run in a hundred	Latches to line threads up; repeat; stress in a separate job

- **Clock.fixed** makes "now" a constant; production passes `Clock.systemUTC()`.
- **new Random(42)** gives the same sequence on every run and every machine.
- **@TempDir** — no leftovers between runs, no clashes between parallel tests.
- **CountDownLatch** releases 16 threads together so they really overlap: 8,000 concurrent saves, none lost.
- **assertTimeoutPreemptively** stops a hung test instead of hanging the whole build.
- For async code, poll for a condition with **Awaitility** (`await().atMost(5, SECONDS).until(…)`) instead of sleeping.

RULE OF THUMB

Treat a flaky test like a bug. Quarantine it (a separate, non-blocking job), open a ticket, and fix or delete it within days — never add an automatic "retry until green" to the main build.

CHAPTER FIVE

05 Beyond correctness

Passing unit and integration tests prove the parts are right. This chapter checks the whole system from the outside: real browsers clicking real pages, real load on real servers, and an attacker's view of your code.

End-to-end & UI testing

TOOL	BROWSERS	NOTES	LATEST
Playwright for Java	Chromium, Firefox, WebKit	Auto-waits for elements; traces with screenshots; one API for browsers and HTTP	Microsoft · 1.63
Selenium WebDriver	Every major browser	W3C WebDriver standard; Selenium Grid runs tests in parallel across machines	4.51
Cypress	Chromium-family, Firefox, WebKit (exp.)	Runs inside the browser; tests are JavaScript/TypeScript only — not Java	JS/TS
REST Assured	— (HTTP)	Java DSL for end-to-end API checks: `given().when().get().then()`	6.1

Playwright for Java · Page Object sketch — not run here

```
class CheckoutPage {                        // a Page Object
  private final Page page;
  CheckoutPage(Page page) { this.page = page; }

  void addToCart(String product) {
    page.getByRole(AriaRole.BUTTON,
        new Page.GetByRoleOptions().setName("Add " + product)).click();
  }
  void checkout() { page.getByTestId("checkout").click(); }
  Locator total() { return page.getByTestId("order-total"); }
}

@Test
void bookOrderShowsDiscountedTotal() {
  var checkout = new CheckoutPage(page);
  checkout.addToCart("Book");
  checkout.checkout();
  assertThat(checkout.total()).hasText("₹1,140.00"); // retries until true
}
```

Choosing locators (Playwright's order)

1. **Role + name** · `getByRole(BUTTON, name "Checkout")` — what users and screen readers see
2. **Text** · `getByText("Order confirmed")` — user-visible, but changes with copy
3. **Label / placeholder** · `getByLabel("Email")` for form fields
4. **Test id** · `getByTestId` reads `data-testid` by default — stable, invisible to users
5. **CSS / XPath** · `div.cart > span:nth-child(3)` — last resort; breaks on any layout change

A sketch: it needs a running web front end, which this guide's shop example doesn't have. The Playwright calls shown (`getByRole` , `getByTestId` , `PlaywrightAssertions.assertThat(locator).hasText`) are from Playwright's Java API.

Page Objects
Wrap each page in a class with intent-revealing methods. When the HTML changes, you fix one class, not fifty tests.

Never sleep
Wait for a condition, not a duration. Playwright's locators and web-first assertions retry automatically until a timeout.

Keep them few
Cover the journeys that make money or lose data — sign-up, checkout, payment. Everything else belongs lower down.

Own your data
Create users and products through the API at the start of each test; don't depend on what is already in the environment.

Accessibility
Run axe-core (Deque publishes Java bindings for Selenium and Playwright) on key pages; it flags missing labels and poor contrast automatically.

Smoke after deploy
A handful of read-only end-to-end checks against production right after each release catches broken config fast.

Traces, not guesses
Record Playwright traces (DOM snapshots, network, console) on failure and keep them as CI artefacts.

Visual regression
Screenshot comparison catches layout breaks assertions miss; mask dates and other changing areas.

Stable environment
Run against a dedicated environment with known data and stubbed third parties (payments, e-mail).

05 · BEYOND CORRECTNESS

Performance testing

Kinds of performance test

Load	Expected peak traffic, held steady	Does it meet the latency target at normal peak?
Stress	Ramp beyond the peak until it breaks	Where is the limit, and does it fail gracefully?
Spike	A sudden jump: 10× in seconds	Do autoscaling, queues and pools cope?
Soak	Normal load for hours	Memory leaks, connection leaks, growing queues
Scalability	Repeat with 1, 2, 4 instances	Does capacity grow with hardware?

Tools

Gatling	Java, Kotlin or Scala DSL; load tests as code; HTML reports	**3.16**
Apache JMeter	GUI to build test plans, CLI to run them; many protocols	**5.6.3**
Grafana k6	Tests in JavaScript; single binary; good CI fit	**2.3**
JMH	Microbenchmarks for one method; handles JIT warm-up and dead-code elimination	**1.37**

RULE OF THUMB

Report **percentiles**, not averages. p99 = 99% of requests were at least this fast; the slowest 1% is what your busiest users feel on every page that makes many calls.

A real load test — and what it found

LoadSmoke.java — virtual users against OrderApi

```
try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
  for (int u = 0; u < users; u++) {
    pool.submit(() -> {
      while (System.nanoTime() < end) {
        long t0 = System.nanoTime();
        try {
          var res = http.send(req, HttpResponse.BodyHandlers.ofString());
          if (res.statusCode() != 200) errors.inc();
        } catch (Exception e) { errors.inc(); }
        if (t0 > warmUpEnd) latencies.add(System.nanoTime() - t0);
      }
      return null;
    });
  }
}
```

real output · 50 users · 2 CPUs

```
users=50  requests=11,065  errors=0
throughput=1,107 req/s
p50     44.11 ms
p90     48.02 ms
p95     48.72 ms
p99     59.71 ms
p99.9   78.56 ms
max     93.98 ms
mean    45.17 ms
```

RUN (10 S MEASURED, AFTER 3 S WARM-UP)	REQ/S	P50 MS	P95 MS	P99 MS	MAX MS	MEAN MS
1 user	22	44.07	48.11	50.09	51.76	44.76
10 users	224	44.05	48.10	49.40	55.90	44.58
50 users	1,107	44.11	48.72	59.71	93.98	45.17
1 user · TCP_NODELAY	2,890	0.25	0.68	2.18	15.79	0.34
50 users · TCP_NODELAY	6,538	6.87	15.21	21.81	48.38	7.64

The finding. Every request took about 44 ms — even with a single user, so it wasn't load. Our handler sends the headers and the body as two separate small writes; with Nagle's algorithm on, the second waits for the client's delayed ACK — the classic Nagle/delayed-ACK stall. Starting the JVM with `-Dsun.net.httpserver.nodelay=true` (sets `TCP_NODELAY`) cut the single-user p50 from 44 ms to 0.25 ms. No unit test could have found this.

Little's Law checks your numbers: users in flight = throughput × time in system. 50 users ÷ 1,107 req/s = 45.2 ms (measured mean 45.2 ms), and 50 ÷ 6,538 = 7.6 ms (measured 7.6 ms). If your tool's numbers don't satisfy it, the load generator itself is the bottleneck. These figures come from this guide's 2-CPU sandbox: compare runs, not absolute numbers.

WATCH OUT

Coordinated omission: a closed-loop tool like this one stops sending while the server stalls, so slow periods produce fewer samples and percentiles look better than reality. Open-model tools (Gatling's `constantUsersPerSec`, k6 arrival-rate executors) keep sending at a fixed rate.

05 · BEYOND CORRECTNESS

Security & resilience testing

Security testing is mostly automation you add to the pipeline once, plus security cases you write like any other test. The scanners find known problems; your own tests prove your rules — who may see what — actually hold.

KIND	WHAT IT DOES	JAVA-FRIENDLY TOOLS	WHEN
SAST	Reads source or bytecode for dangerous patterns	SpotBugs + Find Security Bugs, Semgrep, CodeQL	Every build
SCA	Finds libraries with known vulnerabilities (CVEs)	OWASP Dependency-Check, Dependabot, Renovate	Every build + daily
Secrets	Finds keys and passwords committed to git	gitleaks, GitHub secret scanning	Every push
DAST	Attacks the running app from outside	ZAP (now "ZAP by Checkmarx"), Burp Suite	Nightly, on staging
Fuzzing	Feeds generated, malformed input to find crashes	Jazzer (coverage-guided, JUnit integration)	Nightly / continuous
Pen test	Humans think like attackers	Internal or external security team	Before big releases

Security tests you write yourself

OrderApiTest.java — hostile inputs ✔ passes

```
@Test void pathTricksAreJustNotFound() throws Exception {
  for (String path : List.of("/orders/..%2f..%2fetc%2fpasswd",
                             "/orders/ord-1'%20OR%20'1'='1",
                             "/orders/" + "x".repeat(5_000))) {
    var res = send("GET", path);
    assertThat(res.statusCode()).isEqualTo(404);
    assertThat(res.body())              // no stack traces, no internals
        .isEqualTo("{\"error\":\"order not found\"}");
  }
}
```

Path traversal, SQL-looking IDs and a 5,000-character ID all get the same plain 404 — no stack trace, no SQL error, no file contents. This is a normal JUnit test; it runs on every build with the rest of the suite. The cards below are more ideas to turn into tests like this one.

Access control
User A asks for `/orders/{id}` of user B → expect 403 or 404, never 200.

Injection
Names like `' OR 1=1 --` and `<script>` are stored and shown as plain text.

Authentication
Expired, tampered and missing tokens all get 401; lock-out after repeated failures.

Input limits
Huge bodies, deep JSON, 10,000-item lists: rejected quickly, not crashing the server.

Error leakage
Errors return a generic message — no stack traces, SQL or internal hostnames.

Headers
Responses with personal data send `Cache-Control: no-store`; HTTPS-only cookies.

WATCH OUT

OWASP ranks **broken access control** as the top web application risk — for example one user reading another's data by changing an ID. Scanners rarely find it, because they don't know who should see what. Write explicit tests for it on every endpoint that takes an ID.

RULE OF THUMB

Fail the build on **new** high-severity findings, not on the backlog — otherwise the first scan blocks everything and people switch it off.

Resilience: test the failure, not just the success

Timeouts
Point the client at a server that never answers; the call must give up in time.

Slow dependency
Add latency with Toxiproxy (a Testcontainers module); check pools don't exhaust.

Partial failure
Kill one instance or the cache during a load test; errors should stay bounded.

Retries
Make the dependency fail twice then succeed; assert exactly three calls and idempotent effects.

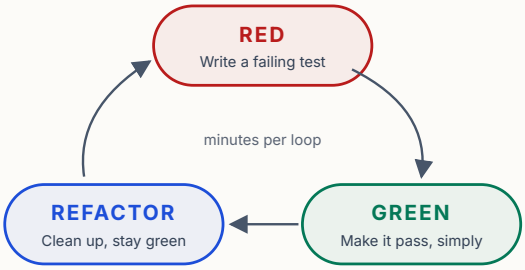
Chaos engineering takes the same idea to production-like environments: inject failures deliberately (kill pods, add latency) and check the system degrades the way you designed it to.

CHAPTER SIX

06 Quality & process

Tests are only as useful as the habits around them: when you write them, how you know they are good enough, and how they run on every change. This chapter covers TDD, coverage, mutation testing, CI — and questions to check your understanding of all of it.

Test-driven development



Kent Beck's *Test-Driven Development: By Example* (2002) describes a short loop: write a test for behaviour that doesn't exist yet, watch it fail, write just enough code to pass, then improve the design while the tests stay green.

- **Red first** proves the test can fail — a test that never failed may not test anything.
- **Simplest code** that passes, even if it looks naive; the next test forces the real logic.
- **Refactor** both code and tests; the tests are your safety net.

A real TDD loop: `PasswordPolicy`

step 1 · tests written, class just compiles

```
public class PasswordPolicy {
  public List<String> problems(String password) {
    return List.of();  // step 1: just enough to compile
  }
}
```

real output · RED — 3 failed

```
JUnit Jupiter
  '-- PasswordPolicyTest [OK]
    +-- passwordWithoutUpperCaseIsReported() [X]
    +-- shortPasswordIsReported() [X]
    +-- strongPasswordHasNoProblems() [OK]
    '-- passwordWithoutDigitIsReported() [X]

shortPasswordIsReported():
  Expecting actual:
    []
  to contain exactly (and in same order):
    ["at least 12 characters"]
  but could not find the following elements:
    ["at least 12 characters"]
```

step 2 · just enough code

```
public class PasswordPolicy {
  public List<String> problems(String password) {
    List<String> problems = new ArrayList<>();
    if (password.length() < 12)
      problems.add("at least 12 characters");
    if (password.chars().noneMatch(Character::isDigit))
      problems.add("a digit");
    if (password.chars().noneMatch(Character::isUpperCase))
      problems.add("an upper-case letter");
    return problems;
  }
}
```

real output · GREEN — 4 / 4 passed

```
PasswordPolicyTest ✔
├─ passwordWithoutUpperCaseIsReported() ✔
├─ shortPasswordIsReported() ✔
├─ strongPasswordHasNoProblems() ✔
└─ passwordWithoutDigitIsReported() ✔
```

Step 3 · refactor: with all four green, we could pull the rules into a list of `(predicate, message)` pairs — and re-run to prove nothing changed.

Honest note: we wrote all four tests before the code, so three failed at once. Strict TDD adds one test at a time. The strong-password test passed even in the red step, because an empty list happens to be its right answer — a reminder to watch *each* new test fail before trusting it.

BDD: tests in the customer's words

orders.feature · Gherkin example

```
Feature: Order discounts
  Scenario: Orders of ₹1000 or more get 5% off
    Given a cart with 2 books at ₹600 each
    When the customer places the order
    Then the order total is ₹1140.00
    And a confirmation e-mail is sent
```

Behaviour-driven development writes acceptance tests as **Given / When / Then** scenarios that product owners can read. Cucumber-JVM maps each line to a Java step method. Worth it when non-developers really read and write the scenarios; otherwise plain JUnit tests with good names are cheaper.

06 · QUALITY & PROCESS

Coverage & mutation testing

Code coverage (JaCoCo)

Instructions	C0	Java bytecode instructions executed — the finest grain
Branches	C1	Both outcomes of every `if` and `switch` case
Lines		A line counts as covered when at least one of its instructions ran
Complexity		Cyclomatic complexity: paths that would need tests
Methods / classes		Executed at least once

Coverage tells you what the tests **ran**, not what they **checked**. A test with no assertions can reach 100%. Use it to find untested code, not as a score to chase.

Mutation testing (PIT)

A tool changes your code in small ways — a **mutant** — and re-runs the tests. If they still pass, the mutant **survived**: your tests missed a bug. Mutation score = killed ÷ total. The idea goes back to Lipton (1971) and DeMillo, Lipton & Sayward (1978).

CONDITIONALS_BOUNDARY	`<` ↔ `<=`, `>=` ↔ `>`
NEGATE_CONDITIONALS	`==` ↔ `!=`, `<` ↔ `>=`
MATH	`+` ↔ `-`, `*` ↔ `/`
INCREMENTS	`i++` ↔ `i--`
INVERT_NEGS	`-x` → `x`
VOID_METHOD_CALLS	removes a call to a void method
…_RETURNS	empty, null, true/false and 0 returns

A mutant, by hand: does the test suite notice?

WeakDiscountPolicyTest.java

```java
/** 100% line + branch coverage, yet the bug survives. */
class WeakDiscountPolicyTest {
  DiscountPolicy policy = new DiscountPolicy();

  @Test void small() {
    assertEquals(0, policy.percentFor(Money.of("500")));
  }
  @Test void medium() {
    assertEquals(5, policy.percentFor(Money.of("2000")));
  }
  @Test void large() {
    assertEquals(10, policy.percentFor(Money.of("6000")));
  }
}
```

the mutant · CONDITIONALS_BOUNDARY

```
-    if (subtotal.compareTo(Money.of("1000")) >= 0) return 5;
+    if (subtotal.compareTo(Money.of("1000")) > 0) return 5;
```

weak tests vs MUTANT · real output — mutant survived

```
JUnit Jupiter
  '-- WeakDiscountPolicyTest [OK]
    +-- medium() [OK]
    +-- large() [OK]
    '-- small() [OK]
```

boundary tests (p.05) vs MUTANT — mutant killed

```
JUnit Jupiter
  '-- DiscountPolicyTest [OK]
    '-- percentForSubtotal(String, int) [OK]
      +-- ₹0 → 0% off [OK]
      +-- ₹999.99 → 0% off [OK]
      +-- ₹1000 → 5% off [X] expected: <5> but was: <0>
      +-- ₹1000.01 → 5% off [OK]
      +-- ₹4999.99 → 5% off [OK]
      +-- ₹5000 → 10% off [OK]
      '-- ₹250000 → 10% off [OK]
```

The weak tests execute every line and both outcomes of each `if`, so line and branch coverage are both 100% — yet they let the bug through. The boundary test at exactly ₹1000 kills it. PIT does this automatically for thousands of mutants: `mvn test-compile org.pitest:pitest-maven:mutationCoverage` (pitest-maven 1.30.0, with the `pitest-junit5-plugin` for JUnit 5/6 tests).

Test smells to refactor away

SMELL	LOOKS LIKE	FIX
Assertion-free test	Calls code, checks nothing — still counts as coverage	Assert the outcome, or delete the test
Eager test	One test checks five behaviours; the name says “testOrder”	Split it: one behaviour, one name
Mystery guest	Expected values come from a file or DB row you can’t see	Build the data inside the test (builders)
Fragile test	Breaks on harmless refactors; verifies every internal call	Assert results, not implementation steps
Sleepy test	`Thread.sleep` to wait for async work	Wait for a condition with a timeout

RULE OF THUMB

Run mutation testing where it matters — pricing, permissions, money — or only on files changed in a pull request; it re-runs tests for every mutant, so it is slow.

06 · QUALITY & PROCESS

Mutation testing with PIT — for real

We ran **PIT 1.30.0** with **pitest-junit5-plugin 1.2.3** on the shop example: every production class mutated with the DEFAULTS group, every unit and integration test run against each mutant. Below is what it found and how we fixed it.

Run 1 — what PIT reported

real PIT summary · run 1 — 39/40 detected

```
Line Coverage (for mutated classes only): 76/77 (99%)
22 tests examined
Generated 40 mutations Killed 39 (98%)
Mutations with no coverage 0. Test strength 98%
```

"22 tests examined" counts test *classes*; PIT ran each mutant against only the tests that cover its line. **Run 2**, after the fix: 40/40 detected — **100%**. With the larger STRONGER group: 50/50 (100%).

GOOD TO KNOW

Before writing a test for a survivor, ask whether the behaviour matters. Some mutants are **equivalent** — the change can't alter any observable result — and no test can kill them. Accept those rather than writing a meaningless test.

The survivor

OrderApi.stop(), line 24: PIT deleted the call `server.stop(0)` and **all 43 tests still passed**. Nothing checked that stopping the API actually frees its port — a resource leak would ship unnoticed.

the test we added to kill it — ✔ passes

```
/** Kills the mutant PIT found: deleting server.stop(0). */
class OrderApiLifecycleTest {

  @Test void stopReleasesThePort() throws Exception {
    var api = new OrderApi(new InMemoryOrderRepository());
    int port = api.start();

    api.stop();

    assertThatThrownBy(() -> new Socket("127.0.0.1", port).close())
        .isInstanceOf(ConnectException.class);  // port is free
  }
}
```

Results by mutator

MUTATOR	MADE	KILLED	TIMED OUT	SURVIVED · RUN 1	SURVIVED · RUN 2
VOID_METHOD_CALLS	10	5	4	1	0
NEGATE_CONDITIONALS	10	10	0	0	0
NULL_RETURNS	7	7	0	0	0
CONDITIONALS_BOUNDARY	5	5	0	0	0
PRIMITIVE_RETURNS	4	4	0	0	0
EMPTY_RETURNS	3	3	0	0	0
MATH	1	1	0	0	0

What PIT actually changed — a sample of the 40 mutants

CLASS · LINE	PIT'S DESCRIPTION	TESTS RUN	RESULT
DiscountPolicy · 7	changed conditional boundary	12	KILLED
Money · 19	Replaced integer subtraction with addition	8	KILLED
OrderService · 32	removed call to EmailSender::send	6	KILLED
PasswordPolicy · 16	replaced return value with Collections.emptyList	4	KILLED
InMemoryOrderRepository · 15	replaced return value with Optional.empty	5	KILLED
OrderApi · 20	removed call to HttpServer::start	0	TIMED_OUT
OrderApi · 24	removed call to HttpServer::stop	2	KILLED (run 2)

Note what is *not* there: `payments.charge(…)` returns a value, so the DEFAULTS group never removes it — only void calls are removed. Larger groups (STRONGER, ALL) add mutators for those cases.

06 · QUALITY & PROCESS

PIT in your build and CI

pom.xml — the same settings as our run, as Maven config

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.30.0</version>
  <dependencies>
    <dependency>  <!— runs JUnit 5/6 tests —>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.3</version>
    </dependency>
  </dependencies>
  <configuration>
    <targetClasses><param>shop.*</param></targetClasses>
    <targetTests><param>shop.*Test</param></targetTests>
    <mutators><mutator>DEFAULTS</mutator></mutators>
    <mutationThreshold>90</mutationThreshold>  <!— fail below 90% —>
    <coverageThreshold>85</coverageThreshold>
    <withHistory>true</withHistory>           <!— incremental —>
    <threads>4</threads>
    <outputFormats><param>HTML</param><param>XML</param></outputFormats>
  </configuration>
</plugin>
```

Run it

COMMAND	DOES
`mvn test-compile pitest:mutationCoverage`	full analysis; HTML report in `target/pit-reports`
`-DtargetClasses=shop.pricing.*`	narrow the run to one package
`-DwithHistory`	reuse the last run's results (incremental)
`-Dmutators=STRONGER`	more mutators; slower, finds more gaps

Making it fast enough for CI

- **Incremental:** with history on, only changed code and tests are re-analysed.
- **Pull requests:** PIT 1.30 has no git-diff goal. Narrow `targetClasses` to the changed module (or use Arcmutate's commercial add-ons); run everything nightly.
- **Target it:** modules with money, permissions and business rules first.
- Our run: 39 s for 40 mutants on 2 CPUs, because PIT re-runs only the tests covering each mutant.

Reading a mutant's status

STATUS	MEANING	DETECTED?
KILLED	a test failed — the bug was caught	**yes**
TIMED_OUT	mutant made the code hang (e.g. a removed `start()`); counted as detected	**yes**
SURVIVED	every covering test still passed — a gap in your assertions	no
NO_COVERAGE	no test even runs this line	no
NON_VIABLE	the mutated bytecode was invalid	—
MEMORY_ERROR / RUN_ERROR	the test JVM crashed	**yes**

Unit or integration tests?

PIT runs whatever tests you point it at. Ours included the HTTP integration tests, which is how it could detect mutants in `OrderApi` at all — no unit test touches that class. The cost: mutants that broke `start()` or the handler made tests hang until PIT's timeout (the 4 **TIMED_OUT** results).

A practical split: run PIT on every pull request with unit tests only (fast, precise), and nightly with integration tests included for the modules where wiring matters. Keep those tests quick — random ports, fakes, no sleeps.

A workflow for survivors

1 · Read it
Open the HTML report: the line, the change PIT made, and which tests covered it.

2 · Does it matter?
Would a user, caller or log notice the change? If not, it may be equivalent or the code may be dead.

3 · Test or delete
Write the assertion that fails on the mutant — or delete code that nothing depends on.

4 · Gate the trend
Fail the build when the score drops below the threshold, so it can't quietly erode.

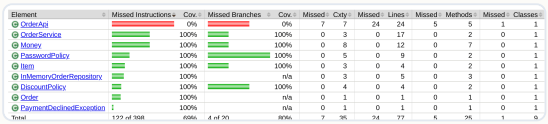
06 · QUALITY & PROCESS

Coverage for unit *and* integration tests

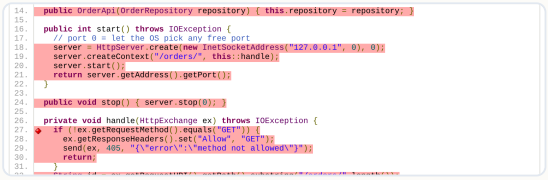
By default most builds measure coverage for unit tests only, so code exercised only by integration tests looks untested. We ran the **JaCoCo 0.8.15** agent twice — once for the 39 unit tests, once for the 5 API integration tests — and merged the results (real numbers from `coverage.sh`; classes without branches left out).

CLASS	UNIT LINES	IT LINES	MERGED LINES	UNIT BRANCHES	IT BRANCHES	MERGED BRANCHES
OrderService	100%	0%	100%	100%	0%	100%
DiscountPolicy	100%	0%	100%	100%	0%	100%
Money	100%	42%	100%	100%	50%	100%
PasswordPolicy	100%	0%	100%	100%	0%	100%
Item	100%	75%	100%	100%	50%	100%
OrderApi	0%	100%	100%	0%	100%	100%
TOTAL	**69%**	**49%**	**100%**	**80%**	**30%**	**100%**

- **Unit tests alone: 69% of lines** — `OrderApi` looks completely untested.
- **Integration tests alone: 49%** — they never touch the pricing or password rules.
- **Merged: 100% of lines and branches.** Each level covers what the other can't.
- Coverage is a floor, not a goal: PIT (p.23) is what showed the tests also *check* what they run.



JaCoCo's own HTML report, unit tests only



Red lines: never executed by unit tests — only the HTTP integration tests reach them

pom.xml — unit + IT coverage, merged, with a gate

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.15</version>
  <configuration>  <!-- report and check read the merged file -->
    <dataFile>${project.build.directory}/jacoco-merged.exec</dataFile>
  </configuration>
  <executions>
    <execution><id>unit</id>       <!-- for Surefire -->
      <goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>it</id>         <!-- for Failsafe -->
      <goals><goal>prepare-agent-integration</goal></goals></execution>
    <execution><id>merge</id><phase>verify</phase>
      <goals><goal>merge</goal></goals>
      <configuration>
        <fileSets><fileSet>
          <directory>${project.build.directory}</directory>
          <includes><include>jacoco.exec</include>
                    <include>jacoco-it.exec</include></includes>
        </fileSet></fileSets>
        <destFile>${project.build.directory}/jacoco-merged.exec</destFile>
      </configuration></execution>
    <execution><id>report</id><phase>verify</phase>
      <goals><goal>report</goal></goals></execution>
    <execution><id>gate</id><phase>verify</phase>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules><rule><element>BUNDLE</element><limits><limit>
          <counter>BRANCH</counter><value>COVEREDRATIO</value>
          <minimum>0.80</minimum>
        </limit></limits></rule></rules>
      </configuration></execution>
  </executions>
</plugin>
```

How the pieces fit

- **prepare-agent** sets `argLine` for Surefire; **prepare-agent-integration** does the same for Failsafe, writing `jacoco-it.exec`.
- **merge** combines the `.exec` files; **report** turns them into HTML, XML and CSV.
- **check** fails the build when a rule isn't met — here 80% of branches.
- Multi-module builds: **report-aggregate** in a separate reporting module.
- Other agents in `argLine` (like the Mockito agent on p.13) must keep JaCoCo's value: write `@{argLine} -javaagent:…`.

RULE OF THUMB

Gate coverage on **new or changed code** (SonarQube's "new code" quality gate, or a diff-coverage tool) rather than the whole code base. Old, untested code then can't block every merge, but nothing new arrives untested.

06 · QUALITY & PROCESS

Finding duplicate & redundant tests

Two tests duplicate each other when they **catch the same bugs**. Run PIT with `fullMutationMatrix=true` — it records every test that kills each mutant — then compare the tests' kill sets. The comparison is a small Java class with no dependencies; Maven or Gradle runs it right after PIT.

src/tools/shop/RedundantTests.java — comparing kill sets

```
List<String> tests = new ArrayList<>(kills.keySet());
tests.sort(Comparator.comparing((String t) -> -kills.get(t).size())
    .thenComparing(Comparator.naturalOrder()));
Map<Set<Integer>, List<String>> groups = new LinkedHashMap<>();
for (String t : tests)
  groups.computeIfAbsent(kills.get(t), k -> new ArrayList<>()).add(t);
for (List<String> g : groups.values())
  if (g.size() > 1)
    System.out.println("  " + String.join("  ≡  ", g) + "   ("
        + kills.get(g.get(0)).size() + " mutants)");

for (String a : tests)
  for (String b : tests)
    if (!a.equals(b) && kills.get(b).containsAll(kills.get(a))
        && kills.get(b).size() > kills.get(a).size()) {
      System.out.printf("  %s (%d) ⊂ %s (%d)%n", a,
          kills.get(a).size(), b, kills.get(b).size());
      break;
    }
```

real output · identical from Maven and Gradle

```
39 tests kill at least one mutant

Same mutants killed:
  DiscountPolicyTest.percentForSubtotal#4  ≡
DiscountPolicyTest.percentForSubtotal#5   (6 mutants)
  ItemTest.ValidQuantity.totalIsPriceTimesQty#2  ≡
ItemTest.ValidQuantity.totalIsPriceTimesQty#3   (5
mutants)
  PasswordPolicyTest.passwordWithoutDigitIsReported  ≡
PasswordPolicyTest.passwordWithoutUpperCaseIsReported
(5 mutants)
  OrderApiTest.pathTricksAreJustNotFound  ≡
OrderApiTest.unknownOrderReturns404   (4 mutants)
  ItemTest.InvalidQuantity.rejected#1  ≡
ItemTest.InvalidQuantity.rejected#2  ≡
ItemTest.InvalidQuantity.rejected#3  ≡  OrderApiTest
(2 mutants)
  MockitoFeaturesTest.consecutiveAnswersForRetries  ≡
MockitoFeaturesTest.finalClassesCanBeMockedByDefault
≡  MoneyTest.rejectsNegativeAmounts   (1 mutants)

Strict subset of another test: 29 tests
```

pom.xml · mvn test-compile pitest:mutationCoverage exec:java

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.6.4</version>
  <configuration>
    <mainClass>shop.RedundantTests</mainClass>
    <classpathScope>test</classpathScope>
    <arguments>
      <argument>target/pit-reports/mutations.xml</argument>
    </arguments>
  </configuration>
</plugin>
```

build.gradle.kts · gradle pitest findRedundantTests

```
tasks.register<JavaExec>("findRedundantTests") {
  group = "verification"
  classpath = sourceSets["tools"].runtimeClasspath
  mainClass = "shop.RedundantTests"
  args("build/reports/pitest/mutations.xml")   // from the pitest
task
  jvmArgs("-Dstdout.encoding=UTF-8")
}
```

WATCH OUT

Both builds ran on GitHub Actions (Apache Maven 3.10.0, Gradle 9.8.1, JDK 21.0.12.1): PIT killed 40 of 40 mutants and the checker printed the same report in both. Equal kill sets mean the tests *currently* catch the same bugs — not that they are the same test. Delete only when the inputs are in the same partition *and* the assertions are the same.

A human verdict on each match

MATCH	WHAT THE TESTS ARE	VERDICT
ItemTest…rejected #1 ≡ #2 ≡ #3	quantities 0, -1, MIN_VALUE	**Redundant** — same partition. Keep 0 (the boundary) and one negative.
DiscountPolicyTest #4 ≡ #5	₹4999.99 and ₹5000	**Keep both** — the two sides of a boundary.
PasswordPolicyTest digit ≡ upper-case	two different rules	**Keep both** — they specify different rules.
OrderApiTest pathTricks ≡ unknownOrder	both expect 404	**Merge candidate** — the hostile-input loop already covers the plain 404.
Unit test ⊂ OrderServiceTest	e.g. `MoneyTest`	**Keep** — a failing broad test doesn't say *which* rule broke.

06 · QUALITY & PROCESS

Build plugins that guard test quality

Everything in this chapter can run inside the build, so quality is checked on every change rather than when someone remembers. Versions are the latest releases at the time of writing.

CONCERN	MAVEN PLUGIN	GRADLE PLUGIN	NOTES
Unit tests	maven-surefire-plugin **3.6.0**	built-in `test` task	`Test*`, `*Test`, `*Tests`, `*TestCase`
Integration tests	maven-failsafe-plugin **3.6.0**	a second `Test` task	`IT*`, `*IT`, `*ITCase`; fails in `verify`
Extra test source dirs	build-helper-maven-plugin **3.6.2**	`sourceSets` (built-in)	e.g. `src/it/java`
Coverage	jacoco-maven-plugin **0.8.15**	`jacoco` (built-in)	gate: `check` / `jacocoTestCoverageVerification`
Mutation testing	pitest-maven **1.30.0**	`info.solidsoft.pitest` **1.19.0**	gate: `mutationThreshold`
Copy-paste in tests	maven-pmd-plugin **3.28.0** · `cpd-check`	`de.aaschmid.cpd` **3.5** · `cpdCheck`	both use PMD's CPD engine
Tests that assert nothing	maven-pmd-plugin · `check`	`pmd` (built-in)	rule `UnitTestShouldIncludeAssert`
Bug patterns	spotbugs-maven-plugin **4.10.5.0**	`com.github.spotbugs` **6.5.13**	null dereferences, bad `equals`, races
Code style	maven-checkstyle-plugin **3.6.0**	`checkstyle` (built-in)	keeps test code readable too
Vulnerable dependencies	dependency-check-maven **13.0.0**	`org.owasp.dependencycheck` **13.0.0**	OWASP Dependency-Check (SCA)
Flaky-test retries	Surefire `rerunFailingTestsCount`	`org.gradle.test-retry` **1.6.6**	reports a pass-on-retry as flaky
Quality gate & history	sonar-maven-plugin **5.8.0.7211**	`org.sonarqube` **7.5.0.8588**	coverage and duplication on new code

Copy-paste detection in test code — a real run

pom.xml · mvn generate-test-sources pmd:cpd-check

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.28.0</version>
  <configuration>
    <includeTests>true</includeTests>
    <minimumTokens>25</minimumTokens>
    <printFailingErrors>true</printFailingErrors>
  </configuration>
</plugin>
```

build.gradle.kts · gradle cpdCheck

```
cpd {
  toolVersion = "7.17.0"
  minimumTokenCount = 25
}
tasks.cpdCheck {
  reports { text.required = true }
}
```

real output · PMD 7.17.0 · tests that assert nothing

```
$ pmd check -d src/test/java -d src/it/java -R
…/UnitTestShouldIncludeAssert
(no violations, exit code 0)

$ pmd check -d demo/pmd -R …/UnitTestShouldIncludeAssert
demo/pmd/NoAssertTest.java:4:   UnitTestShouldIncludeAssert:
This unit test should include assert() or fail()
exit code 4
```

real output · 8 duplications (Maven: 8)

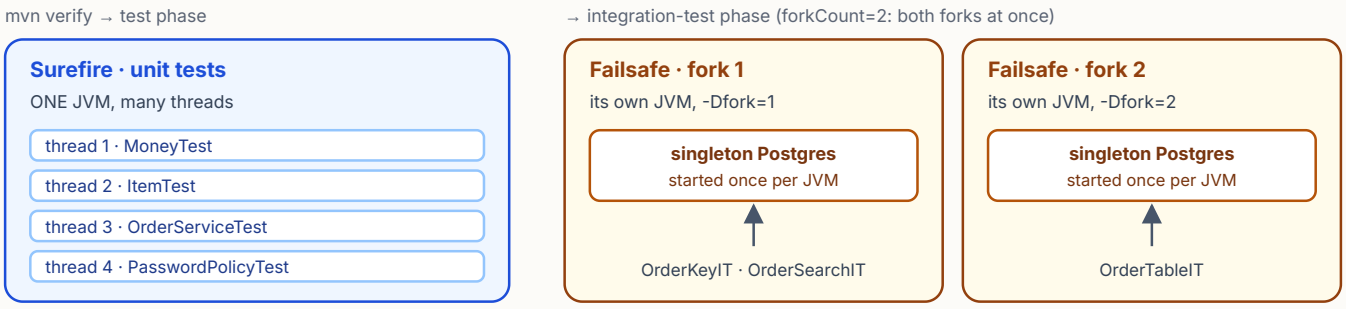
```
Found a 11 line (49 tokens) duplication in the following files:
Starting at line 27 of src/it/java/shop/SharedApi.java
Starting at line 34 of src/it/java/shop/SharedPostgres.java
    log("API started on " + BASE_URL);
  }                          // no stop(): JVM exit frees it
  static void log(String msg) {
    System.out.printf("[fork %s pid %d] %s%n",
        FORK, ProcessHandle.current().pid(), msg);
  }
  /** Unique per fork and test: forks never share rows. */
  static String uniqueId(String test) {
    return "f" + FORK + "-" + test;
```

- **The defaults miss test duplication.** Maven's CPD skips tests unless `includeTests` is on, and needs 100 tokens; the Gradle plugin checks every source set at 50. On this project: **0** duplications at 100 tokens, **0** at 50, **8** at 25.
- The finding above is real: our two singleton base classes (p.29) repeat `log()` and `uniqueId()` — a shared helper fixes it.
- Run alone, `pmd:cpd-check` skips the lifecycle, so folders added by build-helper are missed — our first run found 2, not 8. Run it after `generate-test-sources` or bind it to `verify`.
- Both fail the build on duplicates. PMD's `UnitTestShouldIncludeAssert` rule (left) finds tests that check nothing.
- The Gradle CPD plugin (3.5, Nov 2024) prints Gradle 9 deprecation warnings.

06 · QUALITY & PROCESS

Parallel tests: threads for unit, JVMs for integration

Unit tests share nothing, so they run as **threads in one JVM**. Integration tests share heavy, stateful resources — a database, a container, a port — so they run in **several JVMs**, each with its own copy of those resources.



	UNIT TESTS	INTEGRATION TESTS
Run side by side as	threads in one JVM	separate JVM processes (forks)
What tests share	nothing — objects live in memory	containers, ports, database rows, Spring context, files
Configure with	JUnit `junit.jupiter.execution.parallel.*`	Failsafe `forkCount`; Gradle `maxParallelForks`
Isolation comes from	no shared mutable state; `@ResourceLock` for the rest	each fork has its own JVM, singletons and data
Cost of one more	a thread — almost free	a JVM start and its own containers
Usual setting	on, for every build	a few forks — measure, don't guess (p.29)

pom.xml — Surefire threads, Failsafe forks

```
<!-- UNIT tests: one JVM, many threads -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.6.0</version>
  <configuration>
    <properties>
      <configurationParameters>
        junit.jupiter.execution.parallel.enabled = true
        junit.jupiter.execution.parallel.mode.default = concurrent
        junit.jupiter.execution.parallel.mode.classes.default = concurrent
      </configurationParameters>
    </properties>
  </configuration>
</plugin>

<!-- INTEGRATION tests: many JVMs, one singleton container each -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.6.0</version>
  <configuration>
    <forkCount>${it.forks}</forkCount>
    <reuseForks>true</reuseForks>
    <!-- -Dfork=1, 2, ...: which fork this JVM is -->
    <argLine>@{argLine} -Xshare:off -Dfork=${surefire.forkNumber}
             -javaagent:${org.mockito:mockito-core:jar}</argLine>
  </configuration>
  ...
</plugin>
```

build.gradle.kts — Gradle forks

```
// INTEGRATION tests: many JVMs, one singleton
container each
val integrationTest = tasks.register<Test>
("integrationTest") {
  description = "Runs *IT classes in parallel JVMs"
  group = "verification"
  testClassesDirs =
sourceSets["integrationTest"].output.classesDirs
  classpath =
sourceSets["integrationTest"].runtimeClasspath
  useJUnitPlatform()
  maxParallelForks = 2
  shouldRunAfter(tasks.test)
  testLogging { showStandardStreams = true }
}
tasks.check { dependsOn(integrationTest) }
```

- **forkCount=2** runs up to two test JVMs at once; `2C` means two per CPU core. **reuseForks=true** (the default) keeps each JVM for many classes, so its singletons start once.
- `${{surefire.forkNumber}}` becomes 1…*forkCount*. **Our run:** a system property that was *only* the placeholder arrived empty, while `shop_${{surefire.forkNumber}}` and `argLine` worked. Print it once.
- Surefire's own `parallel` parameter is for JUnit 4 and TestNG; for JUnit Jupiter pass JUnit's settings, as above.
- Gradle: `maxParallelForks` (default 1). Each worker gets a unique `org.gradle.test.worker` property — unique, but *not* 1…N: our runs got workers 2 and 3.
- `reuseForks=false` (Maven) or `forkEvery` (Gradle) recycles JVMs — for leaky tests only; singletons restart too.
- Maven's `-T` (parallel modules) multiplies the forks: 3 modules × `forkCount=2` can mean 6 JVMs and 6 containers.

06 · QUALITY & PROCESS

Singleton containers: one per test JVM

Testcontainers' documented pattern for sharing a container is the **singleton container**: a `static` field in a base class, started in a static initialiser. It starts once when the first subclass loads, every IT class in that JVM reuses it, and the **Ryuk** container removes it when the JVM exits. With forks: **one container per fork**.

src/it/java/shop/SharedPostgres.java (Testcontainers 2.0.5)

```
abstract class SharedPostgres {
  static final String FORK = System.getProperty("fork",
      System.getProperty("org.gradle.test.worker", "1"));
  static final PostgreSQLContainer DB =
      new PostgreSQLContainer("postgres:17-alpine");

  static {
    DB.start();                 // once per JVM, never stop() here
    // … create the schema, log the container id
  }

  static Connection connect() throws SQLException {
    return DriverManager.getConnection(
        DB.getJdbcUrl(), DB.getUsername(), DB.getPassword());
  }
  // log(), uniqueId(), insert() …
}
```

an IT class just extends it

```
class OrderKeyIT extends SharedPostgres {
  @BeforeAll static void hello() {
    log("OrderKeyIT uses " + DB.getJdbcUrl());
  }

  @Test void duplicateIdIsRejected() throws Exception {
    String id = uniqueId("dup");
    try (var c = connect()) {
      insert(c, id, "10");
      assertThatThrownBy(() -> insert(c, id, "20"))
          .isInstanceOf(SQLException.class)
          .extracting(e -> ((SQLException) e).getSQLState())
          .isEqualTo("23505");          // unique_violation
    }
  }
}
```

WATCH OUT

Don't annotate the singleton with `@Container` and don't call `stop()`: the JUnit extension stops a `static` `@Container` after the last test of its class, so the next class would find it gone.

Real run: Failsafe, `forkCount=2`, Docker on GitHub Actions

Apache Maven 3.10.0 · Failsafe 3.6.0 · 2 containers for 4 database IT classes — 10 tests ✔

```
[fork 2 pid 2409] Postgres 3886bb26d2ed on port 32771
[fork 2 pid 2409] OrderKeyIT uses port 32771
[fork 1 pid 2411] Postgres e8fbadf6ae40 on port 32770
[fork 1 pid 2411] OrderTableIT uses port 32770
  ✔ OrderKeyIT — 9.134 s
[fork 2 pid 2409] OrderSearchIT uses port 32771
  ✔ OrderSearchIT — 0.096 s
  ✔ OrderTableIT — 9.671 s
```

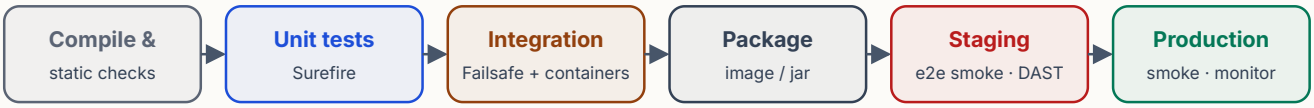
- **One container per JVM.** Fork 1 started a container for `OrderKeyIT` (9.1 s, mostly start-up); `OrderSearchIT` reused it and took 0.10 s. Fork 2 started its own for `OrderTableIT`.
- **Measure it.** Same build, warm Docker, two runs each on a 2-CPU runner: one IT JVM **20.3–20.5 s**, two JVMs **26.5–27.3 s** for the whole `mvn verify`. Two forks were slower here: two containers start at once on a small machine and our tests are short. Forks pay off when IT classes run for minutes.
- **Spring:** the context cache is per JVM too, so each fork builds its own context; wire the container in with `@DynamicPropertySource`.
- Gradle with `maxParallelForks = 2`: the same, one container per worker.

Four ways to share a database

STRATEGY	HOW	TRADE-OFF	USE FOR
Singleton per JVM	one container per fork, shared by its classes	isolation for free; N forks = N containers	**Default choice**
One shared database	one DB; schema `shop_${surefire.forkNumber}` per fork	fewer containers; isolate the data yourself	large suites, heavy images
`@Container` per class	a new container for every test class	simplest, slowest	a few classes only
Reusable containers	`withReuse(true)` + `testcontainers.reuse.enable=true`	survives across runs; experimental	**local only**, not CI

06 · QUALITY & PROCESS

Build tools & CI pipelines



← every push, minutes · fail fast

nightly: full e2e, performance, mutation, DAST →

Maven: Surefire and Failsafe

	SUREFIRE	FAILSAFE
Runs in phase	`test`	`integration-test`, then `verify` fails the build
Default class names	`Test*`, `*Test`, `*Tests`, `*TestCase`	`IT*`, `*IT`, `*ITCase`
Use for	Fast unit tests	Tests needing a started app, DB or containers
Command	`mvn test`	`mvn verify` — so `post-integration-test` clean-up runs

Both are at version 3.6.0. With Gradle, register a second `Test` task (or use the JVM Test Suite plugin) for integration tests.

Run tests in parallel

junit-platform.properties

```
junit.jupiter.execution.parallel.enabled=true
junit.jupiter.execution.parallel.mode.default=concurrent
junit.jupiter.execution.parallel.mode.classes.default=concurrent
junit.jupiter.execution.parallel.config.strategy=dynamic
```

Our 44 tests on 2 CPUs, three runs each: sequential **2,623–2,887 ms**, parallel **1,868–1,989 ms**, all passing. Parallel runs only work if tests share no state — a good test of your tests.

Where should this test live? A Spring Boot service

WHAT YOU'RE CHECKING	LEVEL	TOOLING
Pricing, validation, state rules	`unit`	JUnit + AssertJ, real value objects, no Spring
Service orchestration (charge → save → e-mail)	`unit`	Mockito for ports: gateway, repository, mailer
URL mapping, JSON shape, validation errors, status codes	`integration`	`@WebMvcTest` + `MockMvcTester` + `@MockitoBean`
SQL, mappings, migrations, transactions	`integration`	`@DataJpaTest` + Testcontainers (same DB as prod)
Outbound HTTP client: timeouts, errors, parsing	`integration`	WireMock or `@RestClientTest`
Agreement with another team's service	`contract`	Pact or Spring Cloud Contract
Sign-up, checkout, payment journeys	`end-to-end`	Playwright against staging; a few smoke tests in prod

.github/workflows/build.yml · sketch — not run here

```yaml
name: build
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v6
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: ./mvnw -B verify    # unit + integration
      - uses: actions/upload-artifact@v7
        if: always()             # keep reports on failure
        with: { name: test-reports, path: '**/target/*-reports/*.xml' }
```

Pipeline habits

- **Fastest first.** Static checks and unit tests run before anything slow, so most failures arrive in minutes.
- **Every pull request** runs the full unit and integration suite; merging is blocked on red.
- **Keep reports** (JUnit XML, coverage, Playwright traces) as build artefacts, especially on failure.
- **Budget the time.** If the main build creeps past ~10 minutes, parallelise, split jobs or push slow suites to nightly.
- **Quarantine flaky tests** in a non-blocking job and fix them — never "retry until green".
- **Show results in the pull request:** test, coverage and mutation summaries next to the diff get read; a link to a dashboard doesn't.

06 · QUALITY & PROCESS

Team testing conventions — a starting point

A guide lists options; a team needs decisions. These defaults come from the pages of this guide — each row links to the page that explains it. Copy them into your repository's `CONTRIBUTING.md`, then change what doesn't fit your team. The numbers are suggestions, not standards.

TOPIC	OUR DEFAULT	WHY
Naming & layout		
Unit test classes	`OrderServiceTest` in `src/test/java`, same package as the class	p.30
Integration tests	`OrderFlowIT` — run by Failsafe in `verify`; may need Docker	p.28
Test names	say the behaviour: `unknownOrderIs404`, not `testGet2`	p.04
Shape	one behaviour per test; Arrange · Act · Assert with blank lines between	p.04
Writing tests		
Assertions	AssertJ everywhere; one logical assertion per test, `assertAll`/soft assertions for groups	p.08
Test doubles	mock only the boundaries you own (gateway, repository, mailer); real value objects	p.10
Mockito	`MockitoExtension` (strict stubs); `BDDMockito` in given/when/then tests	p.12
Time & async	inject a `Clock`; no `Thread.sleep` — use Awaitility or latches	p.17
Test data	builders with defaults (`anOrder().withItems(2).build()`); unique IDs per test, so parallel runs never collide	p.28
Spring Boot		
Default level	slices first (`@WebMvcTest`, `@DataJpaTest`); `@SpringBootTest` for a few flows	p.15
Transactions	no `@Transactional` on `@SpringBootTest` tests; one committed-path test per repository	p.16
Database	Testcontainers with the production database and version, schema from the same Flyway/Liquibase migrations — not H2 as a stand-in	p.15
Build gates (suggested numbers — tune them)		
Speed	main build under ~10 minutes; slow suites move to nightly	p.30
Coverage	branch coverage on *new* code ≥ 80% (JaCoCo `check` or a Sonar gate)	p.25
Mutation	PIT on core modules in pull requests, threshold ≥ 80%; everything nightly	p.24
Duplication	CPD on test code too (`includeTests`); start at 50 tokens, lower once clean	p.27
Parallelism	unit tests in threads always; integration forks only after measuring	p.28
Flaky tests	quarantine within a day; fix or delete within the sprint; never retry until green	p.17

Where each check runs

WHEN	CHECKS	ROLE
Every push	unit tests, CPD, PMD, Checkstyle, SpotBugs	minutes; fail fast
Every pull request	integration tests, coverage gate on new code, PIT on the changed module	blocks the merge
Nightly	full PIT, end-to-end and performance suites, Dependency-Check, duplicate-test report	trends and alerts, not gates

Four questions for every test in code review

Would it fail?
If the behaviour broke, would this test go red? (PIT answers this for you.)

Right level?
Is it the lowest level that can catch the bug — unit before integration before end-to-end?

Readable?
Does the name state the expected behaviour, and does a failure message say what went wrong?

Independent?
No shared state, real time, sleeps or test order; safe to run in parallel.

06 · QUALITY & PROCESS

Check your understanding

Twenty questions to check your understanding — for self-review, team discussions or onboarding. Each has a short answer to start from; the index points to the page that covers the topic in depth.

Q01 Unit vs integration test?

A unit test checks one class with its collaborators replaced, in memory and in milliseconds. An integration test checks your code together with real parts — database, HTTP, framework — and is slower but catches wiring and configuration bugs.

Q02 Mock vs stub?

A stub returns canned answers so the code under test can run; you then check the result (state). A mock is set up with expectations about calls, which you verify (behaviour). Mockito objects can play either role.

Q03 What does `@InjectMocks` do?

Creates the class under test and injects `@Mock` / `@Spy` fields by constructor, then setter, then field. It fails silently when it can't inject — many teams prefer calling the constructor explicitly.

Q04 `when().thenReturn()` vs `doReturn().when()`?

Use `doReturn` for spies (the `when` form calls the real method first) and `doThrow` / `doNothing` for void methods.

Q05 How do you test a private method?

Through the public method that uses it. If that is too hard, the private logic probably wants to be its own class with its own tests.

Q06 How do you test code that uses the current time?

Inject a `java.time.Clock`; production passes `Clock.systemUTC()`, tests pass `Clock.fixed(…)`.

Q07 What is a flaky test and how do you fix it?

One that passes and fails without code changes. Find the uncontrolled input — time, randomness, order, shared state, network, sleeps — and control it; quarantine until fixed.

Q08 Is 100% coverage the goal?

No. Coverage shows what ran, not what was checked. Use it to find untested code; use mutation testing to judge test strength.

Q09 `@BeforeEach` vs `@BeforeAll`?

`@BeforeEach` runs before every test on a fresh instance; `@BeforeAll` runs once per class and must be static unless `@TestInstance(PER_CLASS)`.

Q10 Can Mockito mock final classes and statics?

Yes. Since Mockito 5 the inline mock maker is the default, so final classes and methods work; statics need `mockStatic` in a try-with-resources block.

Q11 What is the test pyramid?

Many fast unit tests, fewer integration tests, very few end-to-end tests — because cost and flakiness grow as tests get broader. The testing trophy puts more weight on integration tests.

Q12 How do you test a REST controller in Spring Boot?

`@WebMvcTest` with `MockMvcTester` (or `MockMvc`) and `@MockitoBean` services for the web layer; `@SpringBootTest(webEnvironment = RANDOM_PORT)` plus Testcontainers for a full flow.

Q13 What is TDD and when is it worth it?

Red–green–refactor in short loops. Most valuable for logic with clear rules (pricing, parsing, validation); less so for exploratory UI work.

Q14 How do you test exceptions in JUnit?

`assertThrows(Type.class, () → …)` returns the exception so you can check its message; AssertJ offers `assertThatThrownBy(…)`. Avoid the JUnit 4 `@Test(expected = …)` style — it passes if *any* line throws.

Q15 What is mutation testing?

A tool (PIT) makes small changes to your code — flips `≥` to `>`, removes a call — and re-runs the tests. Surviving mutants show where tests run code without really checking it.

Q16 How do you make tests run in parallel safely?

Remove shared mutable state: no statics, unique test data, temp dirs, random ports. Then enable `junit.jupiter.execution.parallel.enabled`; use `@ResourceLock` for anything truly shared.

Q17 Surefire vs Failsafe?

Surefire runs unit tests in the `test` phase (`*Test`). Failsafe runs integration tests (`*IT`) in `integration-test` and fails the build in `verify`, so clean-up still happens.

Q18 What do you test in a microservice architecture?

Each service with unit and integration tests, the agreements between services with contract tests, and only a few critical journeys end to end.

Q19 How do you find duplicate or redundant tests?

Run PIT with the full mutation matrix and compare which mutants each test kills; equal sets are merge candidates. Add copy-paste detection (PMD CPD, SonarQube) and review — boundary tests often look redundant but aren't.

Q20 What is contract testing?

Consumers publish the requests and responses they rely on; the provider's build verifies them (Pact, Spring Cloud Contract). It replaces many slow end-to-end tests between services.

PIN THIS PAGE

Cheat sheet

Testing Java at a glance

JUNIT 6 ANNOTATIONS

`@Test`	a test
`@ParameterizedTest`	one run per argument set
`@BeforeEach / @AfterEach`	around every test
`@BeforeAll / @AfterAll`	once per class (static)
`@Nested`	group related tests
`@Tag("slow")`	filter what runs
`@Disabled("why")`	skip, with a reason
`@TempDir`	fresh temp folder
`@ExtendWith`	plug in extensions

ASSERTIONS

`assertEquals(exp, act)`	expected first
`assertThrows(X.class, …)`	returns the exception
`assertAll(…)`	report every failure
`assertThat(x).isEqualTo(y)`	AssertJ
`.containsExactly(…)`	order matters
`.extracting(Order::total)`	map, then check
`assertThatThrownBy(…)`	then .hasMessage(...)

MOCKITO

`when(m.f()).thenReturn(v)`	stub
`doReturn(v).when(spy).f()`	stub a spy
`doThrow(e).when(m).voidF()`	stub a void
`verify(m).f(eq(1), any())`	all matchers or none
`verify(m, never()).f()`	not called
`verifyNoInteractions(m)`	untouched
`@Captor ArgumentCaptor<T>`	grab the argument
`mockStatic(X.class)`	try-with-resources

TEST DOUBLES

- **Dummy** — fills a parameter, never used
- **Stub** — canned answers
- **Spy** — stub that records calls
- **Mock** — expectations, verified
- **Fake** — working shortcut (in-memory repo)

WHICH LEVEL?

- **Unit** — rules, calculations, orchestration
- **Integration** — SQL, HTTP, JSON, config
- **Contract** — agreements between services
- **End-to-end** — a few money-making journeys
- Push each check to the **cheapest level** that catches it

FLAKY? CONTROL THE INPUT

- Time → inject `Clock`
- Random → seed it
- Sleep → wait for a condition
- Shared state → fresh fixtures, unique IDs
- Network → stub or container

COMMANDS

`mvn test`	unit tests (Surefire)
`mvn verify`	+ integration tests (Failsafe)
`mvn test -Dtest=MoneyTest`	one class
`mvn test -Dtest='MoneyTest#rejects*'`	matching methods
`mvn test -Dgroups=fast`	only @Tag("fast")
`./gradlew test --tests '*MoneyTest'`	Gradle filter
`mvn test-compile pitest:mutationCoverage`	mutation testing (PIT)
`target/site/jacoco/index.html`	JaCoCo report after mvn verify

GOLDEN RULES

- Watch every new test **fail first**
- One behaviour per test, named as a sentence
- Arrange · Act · Assert — and nothing else
- Test behaviour through the **public API**
- Don't mock values or what you don't own
- Boundaries: test on and either side of every edge
- Coverage finds gaps; **PIT** measures strength — merge unit + IT coverage
- A flaky test is a bug — quarantine and fix it

VERSIONS USED HERE

- JUnit **6.1.3** · Java 17+
- Mockito **5.24.0** · AssertJ **3.27.7**
- Run on OpenJDK **21**

PERFORMANCE

- Report **p50 / p95 / p99**, not the mean
- Warm up the JVM before measuring
- Little's Law: users = rate × time

SECURITY IN CI

- **SAST** + **SCA** + secrets on every build
- **DAST** (ZAP) nightly on staging
- Write access-control tests yourself

BEFORE YOU SHIP

Testing checklist

Unit tests

- [ ] Business rules tested at every boundary
- [ ] Each test checks one behaviour, named as a sentence
- [ ] No real clock, randomness, network or files

Doubles

- [ ] Mocks only at the edges (ports you own)
- [ ] No mocked value objects
- [ ] No unused stubs (strict stubs on)

Integration

- [ ] Real database engine in a container
- [ ] HTTP status, headers and body checked
- [ ] Unhappy paths: 4xx, 5xx, timeouts

Suite health

- [ ] Runs in random order and in parallel
- [ ] No flaky tests in the main build
- [ ] Main build under ~10 minutes

Strength

- [ ] Coverage reviewed for untested code
- [ ] Mutation testing on critical modules
- [ ] Every bug fix starts with a failing test

Beyond

- [ ] A few end-to-end journeys + prod smoke
- [ ] Load test with percentile targets
- [ ] SAST, SCA, DAST and access-control tests

References & further reading

`Beck 2002`	Test-Driven Development: By Example	`Meszaros 2007`	xUnit Test Patterns — the test double names
`Fowler 2007`	Mocks Aren't Stubs — martinfowler.com	`Cohn 2009`	Succeeding with Agile — the test pyramid
`Dodds 2018`	The Testing Trophy — kentcdodds.com	`Robinson 2006`	Consumer-Driven Contracts — martinfowler.com
`DeMillo et al. 1978`	Hints on Test Data Selection — mutation testing	`Winters et al. 2020`	Software Engineering at Google, ch. 11–14
`docs.junit.org`	JUnit 6 user guide & release notes	`site.mockito.org`	Mockito javadoc (incl. agent setup)
`assertj.github.io`	AssertJ core documentation	`docs.spring.io`	Spring Boot 4 testing reference
`java.testcontainers.org`	Testcontainers for Java	`pitest.org · jacoco.org`	Mutation and coverage tools

How this guide was verified

CRITERION	HOW IT WAS CHECKED	RESULT
Code is real	44 unit tests pass on JUnit 6.1.3, Mockito 5.24.0, AssertJ 3.27.7, Java 21. The Maven, Gradle and Spring Boot 4.1 builds — Failsafe forks, PostgreSQL via Testcontainers 2.0.5, Spring slices, JaCoCo, PIT, PMD CPD and the duplicate-test checker — ran on GitHub Actions, and their logs are what those pages show. Every other output comes from one final clean local run.	**Pass**
Sketches labelled	Playwright and the CI sketch need a browser or a pipeline, so they are marked "not run here"; their APIs and versions were checked against official docs. The conventions page states opinions and says so.	**Pass**
Facts & versions	Every version, date, annotation, module name and default checked against official docs, release tags or source code; Mockito strictness, Surefire's fork-number handling and CPD defaults confirmed by running them. Errors found along the way were fixed.	**Pass**
Layout	Automated scan of all 35 pages: no clipped code, no overflowing boxes, no large empty areas; every page also reviewed visually.	**Pass**
Navigation	Clickable contents, page references and footer links (387 links), 41 nested bookmarks, A–Z index.	**Pass**
Accessibility	Tagged PDF, document language, alt text on diagrams and screenshots. Automated WCAG 2.1 AA contrast audit of every text element: 0 failures.	**Pass**
Fonts & text	All 109 font subsets embedded; text, code and ₹ signs search and copy correctly.	**Pass**

FIND ANYTHING

Index A–Z

A

AAA (Arrange · Act · Assert) 04
Acceptance test 03
Access control tests 20
Accessibility testing 18
argLine 25, 13
ArgumentCaptor 11, 12
assertAll 08
AssertJ 08, 09
assertThrows 08
assertTimeoutPreemptively 17
Assumptions 09
Awaitility 17

B

BDD / Gherkin 21
BDDMockito 12
BigDecimal money 08
Boundary value analysis 05, 22
Branch coverage 22

C

Chaos engineering 20
Cheat sheet 33
Classicist vs mockist 11
Clock (java.time) 17, 11
Code coverage 22
Consumer-driven contracts 15
Contract testing 15, 32
Coordinated omission 19
CountDownLatch 17
Coverage: unit vs integration 25
@CsvSource 05, 09
Cucumber 21
Cypress 18

D

DAST 20
@DataJpaTest 15, 16
Decision table 05
Dependency-Check (OWASP) 20, 27
@DisplayName 07
doReturn 13, 12
Dummy 10
Duplicate tests 26

E

End-to-end tests 18
Equivalence partitioning 05
Equivalent mutant 23
@ExtendWith 07, 09

F

Failsafe (Maven) 30, 28
Failure messages 08, 13
Fake 10, 14
FastCSV 06
@FieldSource 09
FIRST principles 04
Flaky tests 17, 30
Flyway / Liquibase 31
forkCount / reuseForks 28
fullMutationMatrix 26
Fuzzing (Jazzer) 20

G

Gatling 19
Given / When / Then 04, 12, 21
Gradle 06, 30

H

Hostile input tests 20
HttpClient (JDK) 14
HttpServer (JDK) 14, 19

I

@InjectMocks 32
InOrder 12
Integration tests 14

J

JaCoCo 22
JaCoCo check (gate) 25
JaCoCo merge / report 25
JMeter 19
JMH 19
JSpecify 06
JUnit 4 migration 06
JUnit 6 changes 06, 09
JUnit Platform 06

K

k6 19
Kill matrix 26

L

LazyInitializationException 16
Lenient stubs 13
Lifecycle 07
Little's Law 19
Load testing 19
Locators 18

M

Maven setup 06
maxParallelForks (Gradle) 28
@MethodSource 09
Mock 10, 11
Mockito 11, 12, 13
Mockito as -javaagent 13
@MockitoBean 15
MockitoExtension 11
MockitoSettings 13
MockMvcTester 15
mockStatic 13
Mutation score 22, 23
Mutation testing (PIT) 22
mutationThreshold 24
Mutator groups (DEFAULTS, STRONGER) 22, 23, 24

N

Nagle's algorithm 19
Naming tests 04
@Nested 09
New-code coverage 25

P

Pact 15
Page Object 18
Parallel execution 30, 28
@ParameterizedTest 05, 09
Percentiles (p95, p99) 19
pitest-junit5-plugin 23, 24
pitest-maven 24, 22
Playwright 18
PMD CPD 27
Port 0 (free port) 14
prepare-agent-integration 25
Property-based testing 05

Q

Quality gate 25, 24
quoteTextArguments 09, 06

R

Record (Java) 07, 13
Redundant tests 26
report-aggregate 25
Resource leak tests 23
REST Assured 18

S

SAST 20
SCA 20
Selenium 18
Self-check questions 32
@ServiceConnection 15
Singleton container 29
Smoke test 03, 18
SonarQube 25, 27
SpotBugs 27
Spring Boot test slices 15, 30
Spy 10, 13
State vs behaviour verification 10
Strict stubs 13
Stub 10
Surefire (Maven) 30, 06
surefire.forkNumber 28, 29
Surviving mutant 23, 24

T

@Tag 09
TCP_NODELAY 19
TDD 21
Team conventions 31
@TempDir 17
Test data builders 31
Test doubles 10
Test pyramid 04
Test sizes (Google) 03
Test smells 22
Testcontainers 15, 14, 29
Testing trophy 04
@TestInstance 07
@TestMethodOrder 07
thenAnswer 11
TIMED_OUT (PIT) 23, 24
Timeouts 09, 17, 20
Toxiproxy 20
@Transactional in tests 16

V

verify 11, 12
Virtual threads 19
Visual regression 18

W

@WebMvcTest 15, 30
WireMock 15
withHistory (PIT) 24

Z

ZAP 20

Three ways to read this guide

New to testing

Foundations **p.03**, first test **p.07**, assertions **p.08**, test doubles **p.10**, Mockito **p.11**, then the cheat sheet **p.33**.

Improving an existing suite

Team conventions **p.31**, flaky tests **p.17**, mutation testing **p.23**, unit + IT coverage **p.25**, duplicates **p.26**, plugins **p.27**, parallel ITs **p.28**, CI **p.30**.

Quick refresher

Pyramid **p.04**, doubles **p.10**, Mockito pitfalls **p.13**, Spring slices **p.15**, TDD **p.21**, Q&A **p.32**.